FIAP ON

DIGITAL BUSINESS ENABLEMENT

PRIMEFACES E AJAX

5

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE FIGURAS

Figura 5.1 – A tela solicitacao.xhtml 14
Figura 5.2 – Primefaces Themes 15
Figura 5.3 – Solicitacao.xhtml com blitzer Theme 16
Figura 5.4 – Create a new resource 18
Figura 5.5 – Create a new Java Class – Theme.java 20
Figura 5.6 – Create a new Java Class – ThemePreferenceService.java 22
Figura 5.7 – Create a new Java Class – TrocaThemeBean.java 24
Figura 5.8 – Solicitacao.xhtml – blitzer Theme 28
Figura 5.9 – Solicitacao.xhtml – black-tie Theme 29
Figura 5.10 – Download Wildfly 11 32
Figura 5.11 – Wildfly 11 Folders 32
Figura 5.12 – Eclipse new server - Wildfly 11 34
Figura 5.13 – Eclipse Install New Software 34
Figura 5.14 – Eclipse Available Software 35
Figura 5.15 – Eclipse Install Remediation Page 36
Figura 5.16 – Eclipse Install Details. 36
Figura 5.17 – Eclipse Review Licenses 37
Figura 5.18 – Eclipse Software Updates 37
Figura 5.19 – Eclipse new server - Wildfly 11 38
Figura 5.20 – Eclipse Create a new server adapter - Wildfly 11 38
Figura 5.21 – Eclipse JBoss Runtime - Wildfly 11 39
Figura 5.22 – Eclipse Server - Wildfly 11 39
Figura 5.23 – index.xhtml 41
Figura 5.24 – Java Resources 41
Figura 5.25 – Eclipse warnings 42
Figura 5.26 – Index.xhtml 45
Figura 5.27 – Project Facets - JPA 45
Figura 5.28 – Ojdbc6.jar 47
Figura 5.29 – Java Resources - JPA 52
Figura 5.30 – Solicitacao.xhtml – Testando o cadastro 56
Figura 5.31 – Solicitacao.xhtml – Resultado do cadastro 56
Figura 5.32 – Oracle Developer TBL_IMV_SOLICITACAO 57
Figura 5.33 – Oracle Developer Comodos e Itens 57
Figura 5.34 – Consulta.xhtml 57
Figura 5.35 – Consulta.xhtml em execução 61
Figura 5.36 – Consulta.xhtml com bootstrap 61

TRIAL

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE QUADROS

Quadro 5.1 – Arquivos 17

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 5.1 – POM.xml 9
Código-fonte 5.2 – Solicitacao.xhtml 13
Código-fonte 5.3 – All-themes 14
Código-fonte 5.4 – Blitzer theme 16
Código-fonte 5.5 – Theme.json 20
Código-fonte 5.6 – Theme.java 21
Código-fonte 5.7 – ThemePreferenceService.java 24
Código-fonte 5.8 – TrocaThemeBean.java 25
Código-fonte 5.9 – Header.xhtml 27
Código-fonte 5.10 – Blitzer theme 27
Código-fonte 5.11 – Configurando o tema dinamicamente 28
Código-fonte 5.12 – TrocaThemeBean.java 30
Código-fonte 5.13 – Standalone.xml 33
Código-fonte 5.14 – Jboss-web.xml 40
Código-fonte 5.15 – POM JSF 2.2.2 40
Código-fonte 5.16 – POM JSF 2.3 40
Código-fonte 5.17– Faces-config.xml 41
Código-fonte 5.18– LinguagemBean.java 42
Código-fonte 5.19 – SolicitacaoBean.java 43
Código-fonte 5.20 – TrocaThemeBean.java 43
Código-fonte 5.21– Init() 44
Código-fonte 5.22 – ThemePreferenceService 44
Código-fonte 5.23 – Persistence.xml 46
Código-fonte 5.24 – POM JPA 47
Código-fonte 5.25 – Solicitacao.java 51
Código-fonte 5.26 – SEQ_SOLICITACAO 52
Código-fonte 5.27 – SolicitacaoDAO.java 53
Código-fonte 5.28 – SolicitacaoDAOImpl.java 53
Código-fonte 5.29 – Trecho do persistence.xml 53
Código-fonte 5.30 – EntityManagerFactorySingleton.java 54
Código-fonte 5.31 – SolicitacaoBean.java 55
Código-fonte 5.32 – GetSolicitacoes() - SolicitacaoBean.java 58
Código-fonte 5.33 – Trecho consulta.xhtml 58
Código-fonte 5.34 – ColumnTogglerdatasource consulta.xhtml 58
Código-fonte 5.35 – Consulta.xhtml 60
Código-fonte 5.36 – Consulta.xhtml 61

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

SUMÁRIO

5 PRIMEFACES E AJAX....7
5.1 Primefaces 7
5.1.1 Primefaces e Ajax 8
5.2 Aplicar o Primefaces no exemplo JSF-Imoveis 8
5.2.1 Reconfigurar o POM do Projeto 8
5.2.2 Adicionar as taglibs e alterar os prefixos 9
5.2.3 Testar o projeto com Primefaces....14
5.2.4 Alterar o tema da nossa aplicação de maneira estática 16
5.2.4.1 Parametrizar o arquivo web.xml 16
5.2.4.2 De maneira dinâmica....17
5.2.4.3 Criar o arquivo themes.json....17
5.2.4.4 Criar a classe Theme 20
5.2.4.5 Criar a classe ThemePreferenceService 22
5.2.4.6 Criar a classe TrocaThemeBean....24
5.2.4.7 Refatorar a View header.xhtml 26
5.2.4.8 Configurar o arquivo web.xml....27
5.2.4.9 Testar a troca dinâmica de temas 28
5.3 Atualizar a aplicação JSF-Imoveis para o JSF 2.3 29
5.3.1 CDI (Contexts and Dependency Injection) 29
5.3.2 Usar anotações no JSF 2.3 30
5.3.3 Os escopos do JavaServer Faces 2.3....31
5.3.4 Recursos necessários 31
5.3.4.1 Baixar e instalar o Wildfly 11 32
5.3.4.2 Configurar o projeto para executar no servidor 33
5.3.4.3 Instalar o JBoss Tools 34
5.3.4.4 Criar um server adapter no projeto para o Wildfly 37
5.3.4.5 Reconfigurar o POM da aplicação....40
5.3.4.6 Reconfigurar o faces-config.xml 41
5.3.4.7 Testar o projeto no Wildfly, backend gerenciado pelo JSF....41
5.3.5 Refatorar os Beans 42
5.3.5.1 O LinguagemBean 42
5.3.5.2 O SolicitacaoBean....42
5.3.5.3 O TrocaThemeBean....43
5.3.5.4 O ThemePreferenceService 44
5.3.6 Testar o projeto no Wildfly, backend gerenciado pelo CDI....45
5.4 A aplicação JSF-Imoveis com JSF 2.3, CDI e JPA 45
5.4.1 Configurando o JPA 45
5.4.2 Configurar o POM do projeto....47
5.4.3 As anotações da Entidade Solicitacao 47
5.4.4 Criar a Sequence SEQ_SOLICITCAO no Oracle FIAP 52
5.4.5 Adicionar as interfaces e classes para persistência 52
5.4.6 A interface SolicitacaoDAO 53
5.4.7 A classe SolicitacaoDAOImpl 53
5.4.8 O Singleton EntityManagerFactorySingleton....53
5.4.9 Refatorar a classe SolicitacaoBean....54
5.4.10 Testar o cadastro 55
5.4.11 Criar a tela de consulta....57

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

5.4.12 Testar a consulta 61
5.5 Considerações finais 62
REFERÊNCIAS 63
GLOSSÁRIO 64

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

5 PRIMEFACES E AJAX

5.1 Primefaces

PrimeFaces é a biblioteca open source mais popular para se criar aplicações JSF. A biblioteca é leve, apesar de possuir mais de 100 componentes. Adicionada com apenas um jar, zero de configuração e sem dependências, na prática, só é necessário baixar o jar do *PrimeFaces*, adicioná-lo ao classpath do projeto e importar o namespace na página xhtml para começar a criar aplicações RIA (Rich Internet Application).

Como utilizamos o Maven, apenas incluiremos uma dependência em nosso POM e configuraremos o repositório do *Primefaces*, para que a biblioteca seja baixada e adicionada à aplicação JSF-Imoveis.

Podemos destacar no Primefaces:

- Mais de uma centena de componentes JSF para interfaces gráficas, como um editor de HTML, galerias de imagens, menus, calendários, charts, mensagens, entre outros;
- Componentes de dados que permitem a implementação personalizada de classificação, filtragem e paginação. Um exemplo desse componente é a tag <p: dataTable />;
- Exemplos detalhados e práticos disponíveis no *Primefaces Showcase*:

https://www.primefaces.org/showcase/

- Suporte refinado para exportar dados das tabelas para outros formatos (XML, XLS, PDF e CSV);
- O framework aporta mais de 35 temas e suporta a criação personalizada;
- Uso melhorado e simplificado da AJAX e suporte para a tecnologia "Push";
- Suporta o desenvolvimento dos aplicativos móveis e possui biblioteca de widgets HTML5.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

5.1.1 Primefaces e Ajax

Ajax (Asynchronous JavaScript and XML) é o uso das tecnologias, tais como JavaScript e XML, que permitem o desenvolvimento das aplicações web mais interativas, capazes de:

- Solicitar e receber dados de um servidor web, após a página ter carregado;
- Atualizar partes de uma página sem precisar recarregar a tela inteira (reload);
- Enviar dados para o servidor web em background;

Apesar do nome, a utilização de XML é opcional e, atualmente, JSON é muito mais empregado, bem como, nem todas as solicitações obrigatoriamente serão assíncronas.

A partir do JSF 2, o suporte a Ajax é fornecido por uma biblioteca JavaScript integrada. Com essa biblioteca, podemos usar Ajax em componentes visuais, como botões, campos, etc.

5.2 Aplicar o Primefaces no exemplo JSF-Imoveis

5.2.1 Reconfigurar o POM do Projeto

Vamos incluir a dependência do *Primefaces*, seu repositório e a dependência do gson que utilizarmos mais à frente na aplicação.

```
<dependencies>
    <dependency>
        <groupId>org.glassfish</groupId>
        <artifactId>javax.faces</artifactId>
        <version>2.2.2</version>
    </dependency>
    <dependency>
        <groupId>org.primefaces</groupId>
        <artifactId>primefaces</artifactId>
        <version>6.2</version>
    </dependency>
    <dependency>
        <groupId>
            org.primefaces.themes
        </groupId>
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
                    <artifactId>all-themes</artifactId>
                    <version>1.0.10</version>
               </dependency>
               <dependency>
                    <groupId>
                         com.google.code.gson
                    </groupId>
                    <artifactId>gson</artifactId>
                    <version>2.8.2</version>
               </dependency>
          </dependencies>
          <repositories>
               <repository>
                    <id>prime-repo</id>
                    <name>
                     PrimeFaces Maven Repository
                   </name>
                    <url>
                     http://repository.primefaces.org
                   </url>
                    <layout>default</layout>
               </repository>
          </repositories>
```

Código-fonte 5.1 – POM.xml
Fonte: Elaborado pelo autor (2018)

5.2.2 Adicionar as taglibs e alterar os prefixos

Na página solicitacao.xhtml, inclua a taglib do *Primefaces* (xmlns:p="*http://primefaces.org/ui*") e altere os prefixos para p: dos componentes que queremos utilizar do *Primefaces*.

Adicione um id para o formulário e altere os componentes <h:selectManyListbox> para <p:selectManyMenu> e <h:inputSecret> para <p:password>.

No commandButton Enviar, adicione o atributo `update="@form"` e remova o atributo `styleClass="submit".` Esse componente tem um visual muito melhor do que o provido pelo Mojarra e já vem com Ajax integrado, o atributo update determina onde a View será atualizada após o submete. @form é igual a `update="formSolicitacao".`

As alterações comentadas acima, estão no seguinte código:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
<!DOCTYPE html>
<ui:composition template="/templates/_template.xhtml"
    xmlns="http://www.w3.org/1999/xhtml"
    xmlns:f="http://xmlns.jcp.org/jsf/core"
    xmlns:h="http://xmlns.jcp.org/jsf/html"
    xmlns:ui="http://xmlns.jcp.org/jsf/facelets"
    xmlns:pt="http://xmlns.jcp.org/jsf/passthrough"
    xmlns:p="http://primefaces.org/ui">

    <ui:define name="title">
        <h:outputText value="#{msg['Index.titulo']}" />
    </ui:define>

    <ui:define name="content">
        <div>
            <h1>#{msg['Solicitacao.titulo']}</h1>
            <h:form id="formSolicitacao">
                <h:messages                id="msgErro"
styleClass="erroMsg" />

                <h:panelGrid              columns="2"
cellpadding="2">

                    <p:outputLabel
value="#{msg['Solicitacao.cidade']}"
                        for="textCidade" />
                    <h:panelGroup layout="block">
                        <p:inputText
id="textCidade" size="30" alt="Cidade"

    value="#{solicitacaoBean.solicitacao.cidade}"
                            autocomplete="off"
                            required="true"

    requiredMessage="#{msg['Solicitacao.cidadeRequerida']}"
/>
                        <p:message for="textCidade"
styleClass="erroMsg" />
                    </h:panelGroup>

                    <p:outputLabel
value="#{msg['Solicitacao.tipo']}" for="selectTipo" />
                    <p:selectOneMenu id="selectTipo"
value="#{solicitacaoBean.solicitacao.tipo}"
                        required="true"

    requiredMessage="#{msg['Solicitacao.tipoRequerido']}">
                        <f:selectItem  itemValue=""
itemLabel="#{msg['Solicitacao.tipoSelecione']}" />
                        <f:selectItems
value="#{solicitacaoBean.tipos}" />
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
                                </p:selectOneMenu>

                                <p:outputLabel
value="#{msg['Solicitacao.operacao']}"
                                     for="oneRadioOp" />
                                <p:selectOneRadio
id="oneRadioOp"

     value="#{solicitacaoBean.solicitacao.operacao}"
required="true"

     requiredMessage="#{msg['Solicitacao.operacaoRequerida']}
">
                                     <f:selectItem
itemLabel="#{msg['Solicitacao.operacao.aluguel']}"
itemValue="Aluguel" />
                                     <f:selectItem
itemLabel="#{msg['Solicitacao.operacao.compra']}"
itemValue="Compra" />
                                </p:selectOneRadio>

                                <p:outputLabel
value="#{msg['Solicitacao.finalidade']}"
                                     for="oneListboxFinalidade"
/>
                                <p:selectOneListbox
id="oneListboxFinalidade"

     value="#{solicitacaoBean.solicitacao.finalidade}"
required="true"

     requiredMessage="#{msg['Solicitacao.finalidadeRequerida'
]}">
                                     <f:selectItem
itemLabel="#{msg['Solicitacao.finalidade.comercial']}"
itemValue="Comercial" />
                                     <f:selectItem
itemLabel="#{msg['Solicitacao.finalidade.residencial']}"
itemValue="Residencial" />
                                </p:selectOneListbox>

                                <p:outputLabel
value="#{msg['Solicitacao.valor']}" for="textValor" />
                                <p:inputText      id="textValor"
size="30"

     value="#{solicitacaoBean.solicitacao.valor}"

     validatorMessage="#{msg['Solicitacao.valorInvalido']}"/>
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
                                <p:outputLabel
for="textareaDesc" value="#{msg['Solicitacao.descricao']}" />
                                <p:inputTextarea
id="textareaDesc" rows="5" cols="40"

     value="#{solicitacaoBean.solicitacao.descricao}" />

                                <p:outputLabel
value="#{msg['Solicitacao.novo']}" for="checkboxNovo" />
                                <p:selectBooleanCheckbox
id="checkboxNovo"

     value="#{solicitacaoBean.solicitacao.novo}" />

                                <p:outputLabel
value="#{msg['Solicitacao.itens']}"   for="manyCheckboxItens"
/>
                                <p:selectManyCheckbox
id="manyCheckboxItens"
value="#{solicitacaoBean.solicitacao.itens}">
                                     <f:selectItem
itemLabel="#{msg['Solicitacao.itens.piscina']}"
itemValue="Piscina" />
                                     <f:selectItem
itemLabel="#{msg['Solicitacao.itens.hidromassagem']}"
itemValue="Hidro" />
                                     <f:selectItem
itemLabel="#{msg['Solicitacao.itens.academia']}"
itemValue="Academia" />
                                     <f:selectItem
itemLabel="#{msg['Solicitacao.itens.sauna']}"
itemValue="Sauna" />
                                </p:selectManyCheckbox>


                                <p:outputLabel
value="#{msg['Solicitacao.comodos']}"
for="manyListboxComodos" />
                                <p:selectManyMenu
id="manyListboxComodos"

     value="#{solicitacaoBean.solicitacao.comodos}">
                                     <f:selectItems
value="#{solicitacaoBean.listaComodos}" />
                                </p:selectManyMenu>

                                <p:outputLabel   for="textEmail"
value="#{msg['Solicitacao.email']}" />
                                <p:inputText id="textEmail"
                                     size="30"
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
     value="#{solicitacaoBean.solicitacao.email}"
                                   required="true"

     requiredMessage="#{msg['Solicitacao.emailRequerido']}"

     validatorMessage="#{msg['Solicitacao.emailInvalido']}" >
                                   <f:validateRegex
                                        pattern="^[_A-Za-z0-9-
\+]+(\.[_A-Za-z0-9-]+)*@[A-Za-z0-9-]+(\.[A-Za-z0-9]+)*(\.[A-
Za-z]{2,})$" />
                              </p:inputText>

                              <p:outputLabel    for="textPass"
value="#{msg['Solicitacao.senha']}" />
                              <p:password id="textPass"
                                   feedback="true"
                                   size="30"

     value="#{solicitacaoBean.solicitacao.senha}" />

                              <p:outputLabel
value="#{msg['Solicitacao.data']}" for="textData" />
                              <p:inputText       id="textData"
readonly="true" size="30"

     value="#{solicitacaoBean.solicitacao.dataSolicitacao.tim
e}">
                                   <f:convertDateTime
pattern="#{msg['Solicitacao.dataFormato']}" />
                              </p:inputText>

                              <p:commandButton value="Enviar"

     actionListener="#{solicitacaoBean.cadastrar}"
update="@form"/>

                              <p:commandButton    value="Home"
immediate="true"
     action="#{solicitacaoBean.navegarHome}" />
                         </h:panelGrid>
                    </h:form>

               </div>
          </ui:define>
       </ui:composition>
```

Código-fonte 5.2 – solicitacao.xhtml
Fonte: Elaborado pelo autor (2018)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

5.2.3 Testar o projeto com Primefaces

Execute a solicitacao.xhtml no Tomcat e a tela será exibida conforme a imagem abaixo. Note a significante melhora no visual.

Figura 5.1 – A tela solicitacao.xhtml
Fonte: Elaborado pelo autor (2018)

Os componentes do *Primefaces* são muito mais ricos em design, quando comparados com o Mojarra. Essa melhora acontece, pois, utilizaram estilos CSS chamados Themes que nós incluímos ao projeto, por meio da dependência do Maven.

```
<dependency>
    <groupId>
            org.primefaces.themes
    </groupId>
    <artifactId>all-themes</artifactId>
    <version>1.0.10</version>
</dependency>
```

Código-fonte 5.3 – all-themes
Fonte: Elaborado pelo autor (2018)

Por padrão, o tema empregado é o Aristo, mas na opção Community (sem custos), existem vários outros temas disponíveis.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

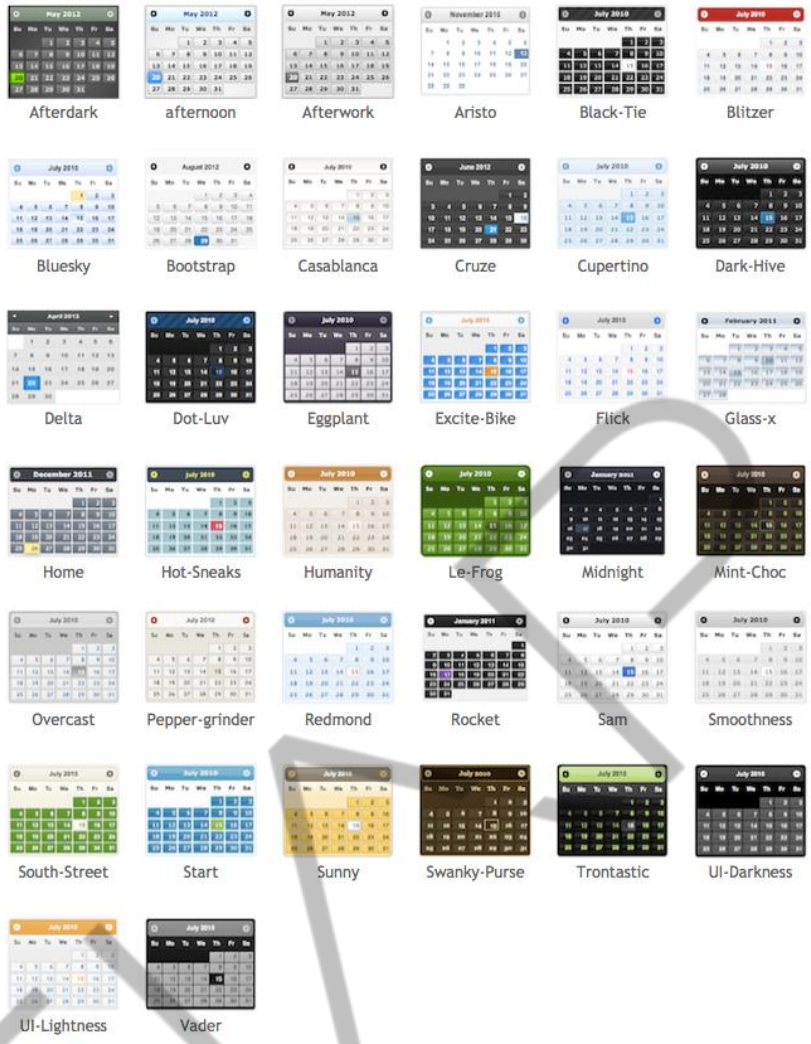

Figura 5.2 – Primefaces Themes
Fonte: Primefaces (2018)

Como dissemos, são várias opções sem custo, mas quando buscar por algo mais avançado, como templates responsivos e temas mais atuais, terá de se tornar um usuário pago ELITE & PRO, para ter acesso ao Premium Omega Application Template ou adquirir um Layout Premium, formado por um conjunto de arquivos xhtml, incluindo painéis, tela de login, dos erros, com bibliotecas de ícones e imagens para agilizar o desenvolvimento.

Cada layout premium oferece um tema compatível com um design semelhante e opções de parametrização das cores e disposições da tela. Para mais informações, acesse https://www.primefaces.org/themes/.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

5.2.4 Alterar o tema da nossa aplicação de maneira estática

5.2.4.1 Parametrizar o arquivo web.xml

Podemos mudar os temas nos componentes do *PrimeFaces*, configurando o novo tema no arquivo **web.xml**. Adicione a configuração abaixo, após a tag que fecha o mapeamento da servlet </servlet-mapping>

```
<context-param>
    <param-name>primefaces.THEME</param-name>
    <param-value>blitzer</param-value>
</context-param>
```

Código-fonte 5.4 – blitzer theme
Fonte: Elaborado pelo autor (2018)

Execute no Tomcat e a tela de solicitações apresentará os componentes com tons de vermelho.

Figura 5.3 – solicitacao.xhtml com blitzer Theme
Fonte: Elaborado pelo autor (2018)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

5.2.4.2 De maneira dinâmica

Para a troca dinâmica, vamos aplicar EL (Expression Language) dentro da tag <param-value /> do arquivo web.xml e utilizar um componente do próprio Primefaces, chamado ThemeSwitcher.

Vamos criar os seguintes arquivos na nossa aplicação:

themes.json	Arquivo Json com todos os temas que serão carregados no componente ThemeSwitcher;
Theme.java	Entidade que será instanciada a cada tema contido no json;
PreferenceThemeService.java	Classe de serviço que lerá o arquivo json e o desserializará para um List<theme> que será retornado para o Managed Bean popular o ThemeSwitcher; Essa classe também será responsável por pegar o primeiro tema e parametrizar a aplicação;
TrocaThemeBean.java	Managed Bean que será responsável por popular o ThemeSwitcher na view header.xhtml e manter na sessão o tema escolhido pelo usuário na troca de uma das opções do componente ThemeSwitcher;

Quadro 5.1 – Arquivos
Fonte: Elaborado pelo autor (2018)

5.2.4.3 Criar o arquivo themes.json

No pacote WebContent/resources/themes, crie um arquivo chamado themes.json. Esse arquivo conterá todos os temas disponíveis.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Eles serão carregados pela classe ThemePreferenceService.java que proverá um List<Theme> para o Managed Bean TrocaThemeBean.java.

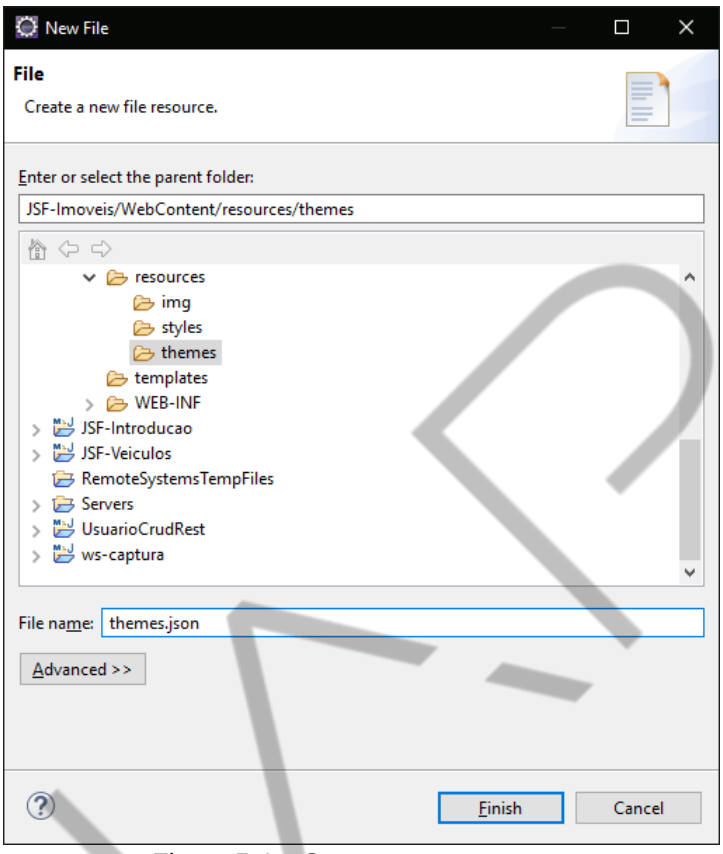

Figura 5.4 – Create a new resource
Fonte: Elaborado pelo autor (2018)

Crie o themes.json com o seguinte código:

```
[
                {"id":0,"displayName":"Blitzer",
"name":"blitzer"},
               {"id":1,"displayName":"Afternoon",
"name":"afternoon"},
               {"id":2,"displayName":"Afterwork",
"name":"afterwork"},
               {"id":3,"displayName":"Aristo",
"name":"aristo"},
               {"id":4,"displayName":"Black-Tie",
"name":"black-tie"},
               {"id":5,"displayName":"Afterdark",
"name":"afterdark"},          {"id":6, "displayName":"Bluesky",
"name":"bluesky"},
               {"id":7,"displayName":"Bootstrap",
"name":"bootstrap"},
               {"id":8,"displayName":"Casablanca",
"name":"casablanca"},
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
                {"id":9,"displayName":"Cupertino",
"name":"cupertino"},
                {"id":10,"displayName":"Cruze",
"name":"cruze"},
                {"id":11,"displayName":"Dark-Hive",
"name":"dark-hive"},
                {"id":12,"displayName":"Delta",
"name":"delta"},
                {"id":13,"displayName":"Dot-Luv",   "name":"dot-
luv"},
                {"id":14,"displayName":"Eggplant",
"name":"eggplant"},
                {"id":15,"displayName":"Excite-Bike",
"name":"excite-bike"},
                {"id":16,"displayName":"Flick",
"name":"flick"},
                {"id":17,"displayName":"Glass-X",
"name":"glass-x"},
                {"id":18,"displayName":"Home", "name":"home"},
                {"id":19,"displayName":"Hot-Sneaks",
"name":"hot-sneaks"},
                {"id":20,"displayName":"Humanity",
"name":"humanity"},
                {"id":21,"displayName":"Le-Frog",    "name":"le-
frog"},
                {"id":22,"displayName":"Midnight",
"name":"midnight"},
                {"id":23,"displayName":"Mint-Choc",
"name":"mint-choc"},
                {"id":24,"displayName":"Overcast",
"name":"overcast"},
                {"id":25,"displayName":"Pepper-Grinder",
"name":"pepper-grinder"},
                {"id":26,"displayName":"Redmond",
"name":"redmond"},
                {"id":27,"displayName":"Rocket",
"name":"rocket"},
                {"id":28,"displayName":"Sam", "name":"sam"},
                {"id":29,"displayName":"Smoothness",
"name":"smoothness"},
                {"id":30,"displayName":"South-Street",
"name":"south-street"},
                {"id":31,"displayName":"Start",
"name":"start"},
                {"id":32,"displayName":"Sunny",
"name":"sunny"},
                {"id":33,"displayName":"Swanky-Purse",
"name":"swanky-purse"},
                {"id":34,"displayName":"Trontastic",
"name":"trontastic"},
```


PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
                {"id":35,"displayName":"UI-Darkness",
"name":"ui-darkness"},
                {"id":36,"displayName":"UI-Lightness",
"name":"ui-lightness"},
                {"id":37,"displayName":"Vader", "name":"vader"}
]
```

Código-fonte 5.5 – theme.json
Fonte: Elaborado pelo autor (2018)

5.2.4.4 Criar a classe Theme

Dentro do pacote entity, crie a classe Theme.java.

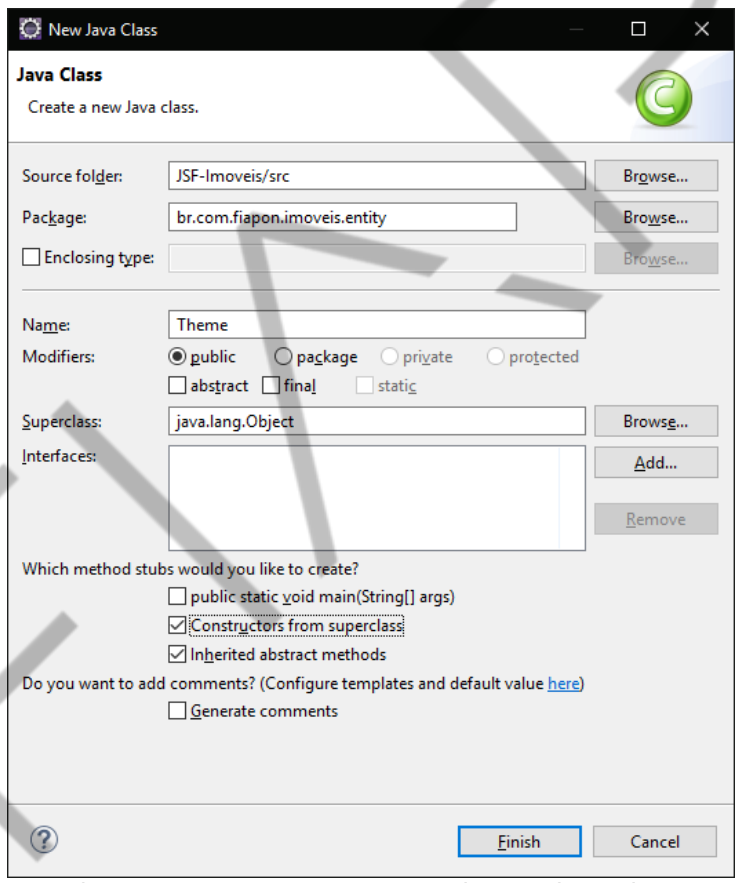

Figura 5.5 – Create a new Java Class – Theme.java
Fonte: Elaborado pelo autor (2018)

Codifique-a, conforme trecho abaixo. Essa entidade será utilizada para armazenar cada tema que temos como opção em all-themes, baixado pelo Maven e parametrizado dinamicamente pelo usuário com o componente ThemeSwitcher.

```
package br.com.fiapon.imoveis.entity;

      import java.io.Serializable;
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
public class Theme implements Serializable{

    private static final long serialVersionUID = 1L;
    private int id;
    private String displayName;
    private String name;

    public Theme() {}

    public Theme(int id, String displayName, String name)
{
        this.id = id;
        this.displayName = displayName;
        this.name = name;
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getDisplayName() {
        return displayName;
    }

    public void setDisplayName(String displayName) {
        this.displayName = displayName;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    @Override
    public String toString() {
        return name;
    }
}
```

Código-fonte 5.6 – Theme.java
Fonte: Elaborado pelo autor (2018)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

5.2.4.5 Criar a classe ThemePreferenceService

No pacote service, crie uma classe chamada ThemePreferenceService.java.

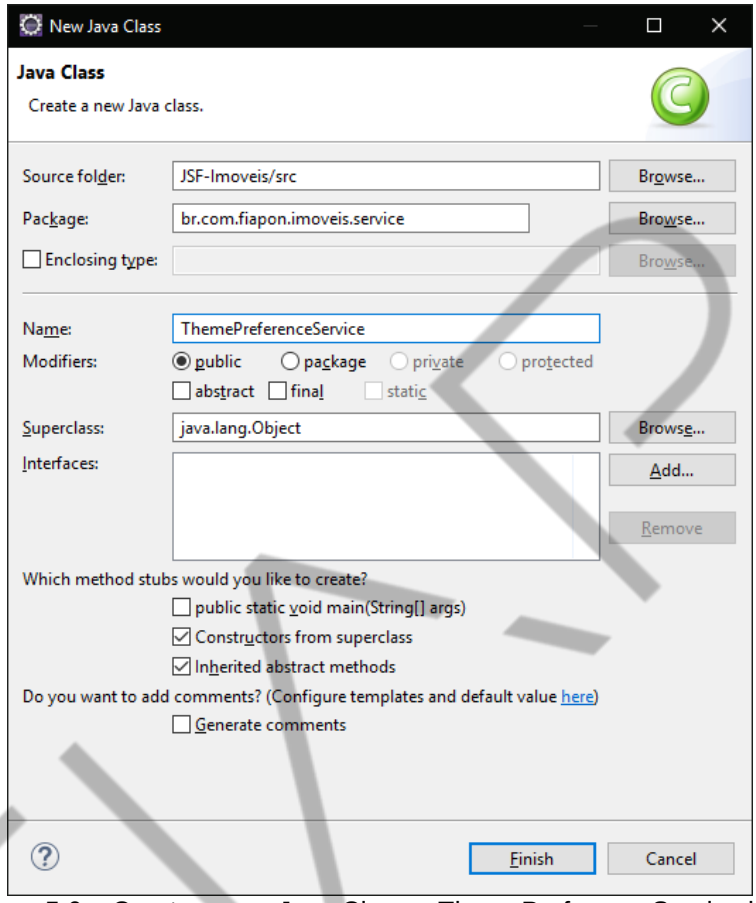

Figura 5.6 – Create a new Java Class – ThemePreferenceService.java
Fonte: Elaborado pelo autor (2018)

Essa classe será responsável por carregar o arquivo themes.json com todos os temas disponíveis e parametrizar a aplicação com o primeiro tema carregado.

O método getThemes() será utilizado pelo Managed Bean para carregar todos os temas disponíveis no componente ThemeSwitcher.

Codifique-a, conforme trecho abaixo.

```
package br.com.fiapon.imoveis.service;

import java.io.IOException;
import java.io.Serializable;
import java.nio.file.Files;
import java.nio.file.Paths;
import java.util.Arrays;
import java.util.List;
import javax.faces.context.FacesContext;
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
import com.google.gson.Gson;
import br.com.fiapon.imoveis.entity.Theme;

public class ThemePreferenceService implements
Serializable{

    private static final long serialVersionUID = 1L;
    private String theme;
    private List<Theme> themes;


    public ThemePreferenceService() {

        try {
            //Busca o path real do deploy no servidor
            String path =
FacesContext.getCurrentInstance().getExternalContext()

.getRealPath("/resources/themes/themes.json");
            //Lê o arquivo themes.json na String json
            String json = new
String(Files.readAllBytes(Paths.get(path)));
            //Desserializa a String json no
List<Theme>
            themes = Arrays.asList(new
Gson().fromJson(json, Theme[].class));
            //Pega o nome do primeiro Theme contido no
List
            //Esse será o Theme padrão ao iniciar a
aplicação
            theme = themes.get(0).getName();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }


    public String getTheme() {
        return theme;
    }


    public void setTheme(String theme) {
        this.theme = theme;
    }


    public List<Theme> getThemes() {
        return themes;
    }
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
        public void setThemes(List<Theme> themes) {
            this.themes = themes;
        }

    }
```

Código-fonte 5.7 – ThemePreferenceService.java
Fonte: Elaborado pelo autor (2018)

5.2.4.6 Criar a classe TrocaThemeBean

No pacote bean, crie uma classe chamada TrocaThemeBean.java.

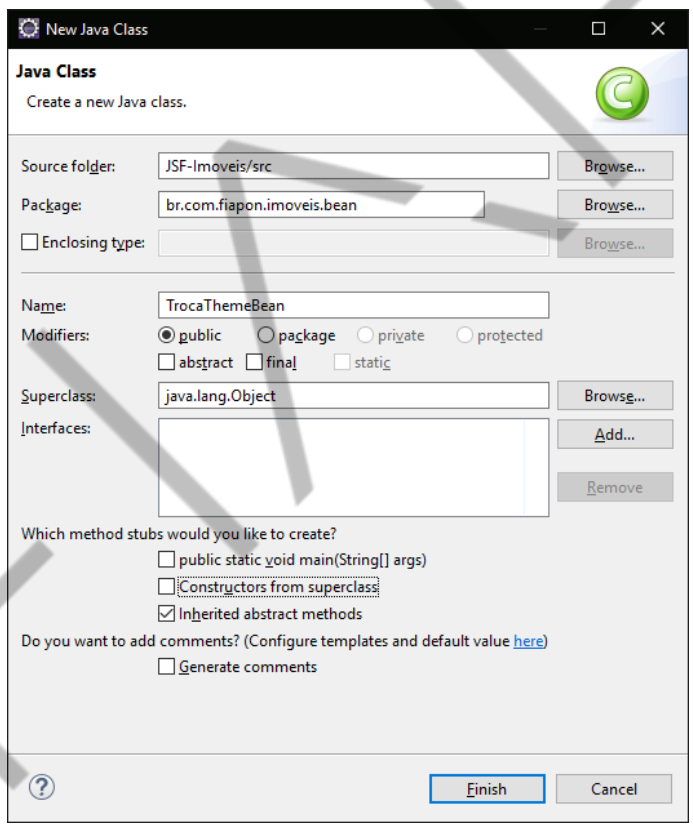

Figura 5.7 – Create a new Java Class – TrocaThemeBean.java
Fonte: Elaborado pelo autor (2018)

Esse Managed Bean será responsável por atender a view header.xhtml que vamos refatorar, para incluir o componente ThemeSwitcher. O Bean, na sua inicialização, buscará os temas disponíveis no service e parametrizará o tema padrão da aplicação, dê especial atenção ao método init().

```
package br.com.fiapon.imoveis.bean;
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
        import java.util.List;

        import javax.annotation.PostConstruct;
        import javax.faces.bean.ManagedBean;
        import javax.faces.bean.SessionScoped;
        import br.com.fiapon.imoveis.entity.Theme;
        import
br.com.fiapon.imoveis.service.ThemePreferenceService;

        @SessionScoped
        @ManagedBean
        public class TrocaThemeBean {

            //atributo com a lista de temas
            private List<Theme> themes;
            //Atributo com o nome do tema corrente
            private String theme;
            private ThemePreferenceService tp;

            public String getTheme() {
                return theme;
            }

            public void setTheme(String theme) {
                this.theme = theme;
            }

            public List<Theme> getThemes() {
                return themes;
            }

            @PostConstruct
            public void init() {
                tp = new ThemePreferenceService();
                //Busca no service todos os temas carregados do
json
                themes = tp.getThemes();
                //Busca do service o tema default
                theme = tp.getTheme();
            }

            public void saveTheme() {
                tp.setTheme(theme);
            }

        }
```

Código-fonte 5.8 – TrocaThemeBean.java
Fonte: Elaborado pelo autor (2018)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

5.2.4.7 Refatorar a View header.xhtml

A view header contém a imagem FiapOn e as bandeiras para a internacionalização. Vamos aproveitar a onde estão as bandeiras e incluir o componente ThemeSwitcher. Não se esqueça de incluir as taglibs necessárias.

Altere a página com o seguinte código:

```
<!DOCTYPE html>

<html xmlns="http://www.w3.org/1999/xhtml"
    xmlns:h="http://xmlns.jcp.org/jsf/html"
    xmlns:ui="http://xmlns.jcp.org/jsf/facelets"
    xmlns:f="http://xmlns.jcp.org/jsf/core"
    xmlns:p="http://primefaces.org/ui">

<h:body>
    <ui:composition>
        <div id="header">
            <h:outputLink     styleClass="headerlink"
value="index.xhtml">
                <h:graphicImage
url="resources/img/fiapon.png" />
            </h:outputLink>
            <h:form>
                <div id="languages">

                    <p:outputLabel   for="tSwitcher"
value="Theme:" />
                    <p:themeSwitcher
value="#{trocaThemeBean.theme}"                id="tSwitcher"
style="width:165px">
                        <f:selectItem
itemLabel="Themes" itemValue="" />
                        <f:selectItems
value="#{trocaThemeBean.themes}" var="theme"

     itemLabel="#{theme.displayName}"
itemValue="#{theme.name}" />
                        <p:ajax
listener="#{trocaThemeBean.saveTheme}" />
                    </p:themeSwitcher>


                    <h:commandLink
action="#{linguagemBean.mudarParaPortugues()}"
                        ajax="false">
                        <h:graphicImage
name="brasil.png" library="img" />
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
                                    </h:commandLink>
                                    <h:commandLink
action="#{linguagemBean.mudarParaIngles()}"
                                        ajax="false">
                                        <h:graphicImage
name="eua.png" library="img" />
                                    </h:commandLink>
                                </div>
                            </h:form>
                        </div>
                    </ui:composition>
                </h:body>
                </html>
```

Código-fonte 5.9 – header.xhtml
Fonte: Elaborado pelo autor (2018)

Perceba que o componente ThemeSwitcher tem como value o atributo theme do Managed Bean e utiliza o p:ajax para chamar o método que troca o tema no Bean. Essa é uma importante funcionalidade do *Primefaces*, o uso de Ajax para invocar um método com o componente p:ajax.

Espero que esteja se perguntando, mas como a aplicação começa com um tema padrão e depois, como esse tema se mantém. Uma importante configuração será feita agora.

5.2.4.8 Configurar o arquivo web.xml

Lembra que escolhemos o tema de maneira estática, adicionar um parâmetro do contexto no arquivo web.xml. Agora, para que o tema fique dinâmico, precisamos substituir o param-value hardcode abaixo:

```
        <context-param>
            <param-name>primefaces.THEME</param-name>
            <param-value>blitzer</param-value>
        </context-param>
```

Código-fonte 5.10 – blitzer theme
Fonte: Elaborado pelo autor (2018)

Pelo código EL seguinte:

```
        <context-param>
            <param-name>primefaces.THEME</param-name>
            <param-value>
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
                    #{trocaThemeBean.theme}
                </param-value>
            </context-param>
```

Código-fonte 5.11 – Configurando o tema dinamicamente
Fonte: Elaborado pelo autor (2018)

Dessa forma, quando a aplicação for carregada pela primeira vez, o tema padrão será configurado e quando o usuário trocar de tema, o web.xml será dinamicamente atualizado.

5.2.4.9 Testar a troca dinâmica de temas

Execute a aplicação e o tema default Blitzer será colocado.

Figura 5.8 – solicitacao.xhtml – blitzer Theme
Fonte: Elaborado pelo autor (2018)

Troque o ThemeSwitcher para Black-Tie e veja o seguinte resultado.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Figura 5.9 – solicitacao.xhtml – black-tie Theme
Fonte: Elaborado pelo autor (2018)

5.3 Atualizar a aplicação JSF-Imoveis para o JSF 2.3

O JSF 2.3 (JavaServer Faces) não perdeu sua essência, continua um framework MVC, baseado em componentes, que nessa versão faz parte do Java EE 8. Os principais recursos novos no JSF 2.3 são uma integração muito maior com CDI, suporte para WebSockets, o uso do component search expression. framework (doado pelo *PrimeFaces*), suporte básico para extensionless URLs e validação em nível de classe Bean.

Podemos dizer que uma mudança importante em termos de anotação foi que os Managed Beans do JSF 2.x ficaram obsoletos no 2.3, a favor do CDI. Agora utilizaremos @Named em vez de @ManagedBean.

O JSF 2.3 (Mojarra 2.3) pode ser baixado por via direta da página de download do projeto ou pelo Maven, veja em https://javaee.github.io/javaserverfaces-spec/ Downloads JSF 2.3.

5.3.1 CDI (Contexts and Dependency Injection)

CDI é um conjunto de serviços que facilitam o uso de Enterprise Beans na programação, em nosso caso, ao utilizarmos em nossos beans JSF. O uso de CDI gera baixo acoplamento entre classes do sistema e permite que seus objetos tenham as dependências fornecidas automaticamente, em vez de criá-los ou recebê-los como parâmetros. O CDI também administra o ciclo de vida dessas dependências.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Um bean anotado como @ManagedBean é instanciado pelo JSF e é executado em um container JSF, em nossos exemplos anteriores, providos ao Tomcat pelo Maven. Já um bean anotado como @Named será instanciado pelo CDI e executado em um container CDI, container que agregará suportes adicionais, indisponíveis no container JSF, tais como, o uso de interceptors, observers, producers, entre outros.

Nesse cenário, o uso de um servidor Full JEE, com todos os containers disponíveis, é mais indicado, portanto, vamos utilizar o Wildfly a partir de agora.

5.3.2 Usar anotações no JSF 2.3

Como comentamos, no JSF 2.3, as anotações Managed Bean estão deprecated e o uso de CDI é a abordagem preferida.

A anotação @Named (javax.inject.Named) em uma classe, juntamente com uma anotação de escopo, registra automaticamente essa classe como um recurso com a implementação JavaServer Faces. Um Bean que usa essas anotações é um Bean gerenciado pelo CDI.

O seguinte trecho de código, mostra um exemplo do uso das anotações @Name e @SessionScoped na classe TrocaThemeBean, que será refatorada mais à frente.

```
import javax.enterprise.context.SessionScoped;
import javax.inject.Named;

@SessionScoped
@Named
public class TrocaThemeBean implements Serializable{
```

Código-fonte 5.12 – TrocaThemeBean.java
Fonte: Elaborado pelo autor (2018)

No JSF 2.3 continuará a usar anotações para definir o escopo no qual o bean será armazenado. Nessa versão, deve especificar um dos escopos abaixo para um bean.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

5.3.3 Os escopos do JavaServer Faces 2.3

Os escopos no JSF 2.3, agora de responsabilidade do CDI, devem ser incluídos do pacote javax.enterprise.context e não mais de javax.faces.bean.

@ApplicationScoped (javax.enterprise.context.ApplicationScoped) é um escopo de aplicação e não de usuário. Portanto, mantém a instância do Bean durante todo o tempo que a aplicação estiver em execução. Por ter um nível de aplicação, pode ser acessado por qualquer página do aplicativo e o que for armazenado em um bean com esse escopo, pode ser acessado de forma compartilhada, por todos os clientes que estejam navegando na aplicação.

@SessionScoped (javax.enterprise.context.SessionScoped) tem a instância mantida enquanto a sessão HTTP existe, geralmente por mais de uma requisição do cliente, para a mesma página ou navegar em mais de uma página. O bean é destruído quando a sessão HTTP termina. Cada cliente possui sua sessão e pode interagir com o que for armazenado.

@FlowScoped (javax.faces.flows.FlowScoped é basicamente o escopo que mantém a instância enquanto o cliente interage com um fluxo, um conjunto de páginas relacionadas a uma lógica de negócios com um ponto de entrada e um ponto de saída. Um carrinho de compras desenvolvido em várias telas pode ser considerado como um bom exemplo de fluxo.

@RequestScoped (javax.enterprise.context.RequestScoped): continua com o tempo de vida enquanto durar uma resposta a uma requisição HTTP.

@DependentScoped (javax.enterprise.context.Dependent) indica que o bean depende de algum outro bean.

5.3.4 Recursos necessários

- JDK 8
- Eclipse Oxygen
- Wildfly 11

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

5.3.4.1 Baixar e instalar o Wildfly 11

A instalação do Wildfly é muito simples, baixar e descompactar. Vamos utilizar a versão 11 pela estabilidade e por ser mais fácil de configurá-la no Eclipse. Em http://wildfly.org/downloads/, selecione a versão 11.0.0 Final. Baixe a distribuição Full compactada e descompacte-a no diretório de sua preferência.

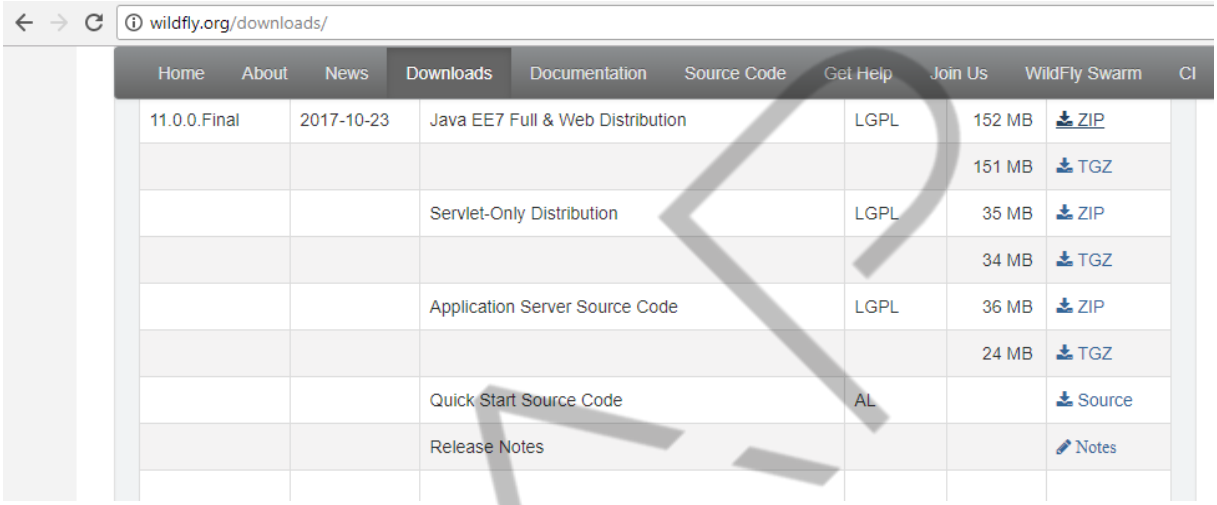

Figura 5.10 – Download Wildfly 11
Fonte: Elaborado pelo autor (2018)

Você terá a seguinte estrutura de diretórios.

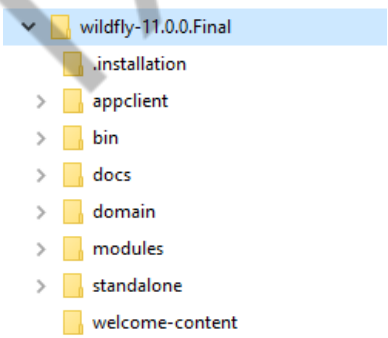

Figura 5.11 – Wildfly 11 Folders
Fonte: Elaborado pelo autor (2018)

Se quiser alterar a porta HTTP do seu servidor, para evitar conflitos, altere o arquivo standalone.xml que fica na pasta standalone\configuration. Eu alterei a porta HTTP para 8089.

```
<socket-binding-group name="standard-sockets" default-interface="public" port-offset="${jboss.socket.binding.port-offset:0}">
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
      <socket-binding                    name="management-http"
interface="management"
port="${jboss.management.http.port:9990}"/>
       <socket-binding                 name="management-https"
interface="management"
port="${jboss.management.https.port:9993}"/>
       <socket-binding                              name="ajp"
port="${jboss.ajp.port:8009}"/>
        <socket-binding                            name="http"
port="${jboss.http.port:8089}"/>
         <socket-binding                          name="https"
port="${jboss.https.port:8443}"/>
        <socket-binding       name="txn-recovery-environment"
port="4712"/>
         <socket-binding            name="txn-status-manager"
port="4713"/>
              <outbound-socket-binding name="mail-smtp">
                  <remote-destination        host="localhost"
port="25"/>
              </outbound-socket-binding>
</socket-binding-group>
```

Código-fonte 5.13 – standalone.xml
Fonte: Elaborado pelo autor (2018)

5.3.4.2 Configurar o projeto para executar no servidor

Na aba server, clique com o botão direito e escolha new server, procure o Wildfly 11. Se o encontrar em server type, salte para o item “Testando o projeto no Wildfly, backend gerenciado pelo JSF” deste documento. Caso contrário, instale o JBoss Tools.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

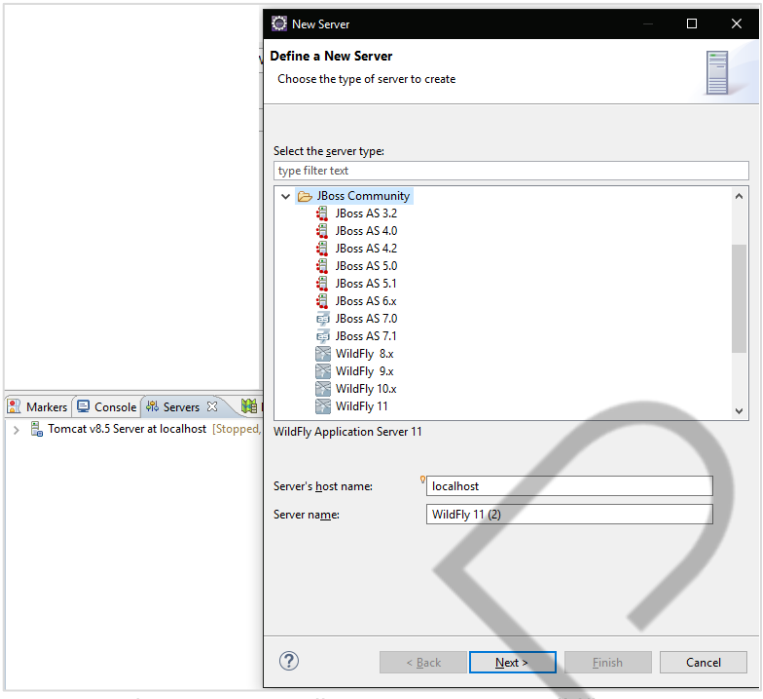

Figura 5.12 – Eclipse new server - Wildfly 11
Fonte: Elaborado pelo autor (2018)

5.3.4.3 Instalar o JBoss Tools

No Eclipse, vá a Help > Install New Software.

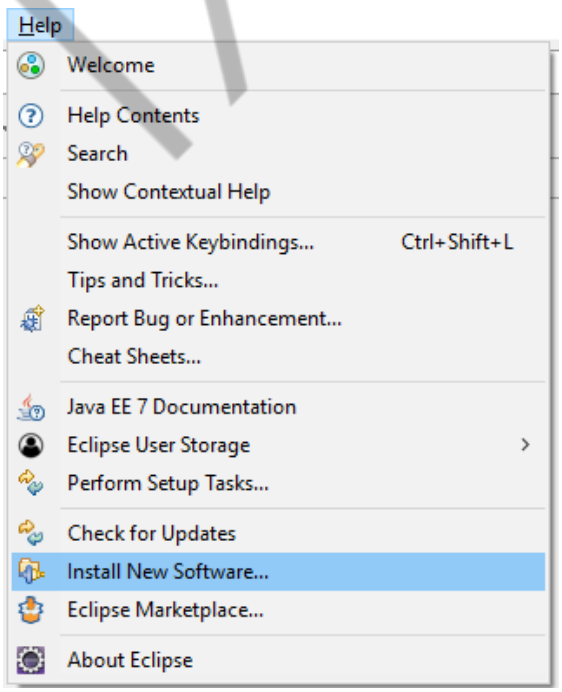

Figura 5.13 – Eclipse Install New Software
Fonte: Elaborado pelo autor (2018)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Caso tenha o Eclipse Oxygen, configure Work with com a URL abaixo e clique em Add...

http://download.jboss.org/jbosstools/oxygen/development/updates/

Caso tenha outra versão do Eclipse, procure a URL na lista abaixo:

Neon: https://download.jboss.org/jbosstools/neon/stable/updates/

Mars: http://download.jboss.org/jbosstools/mars/stable/updates/

Luna: http://download.jboss.org/jbosstools/updates/stable/luna/

Kepler: http://download.jboss.org/jbosstools/updates/stable/kepler/

Juno: http://download.jboss.org/jbosstools/updates/stable/juno/

Indigo: http://download.jboss.org/jbosstools/updates/stable/indigo/

Helios: http://download.jboss.org/jbosstools/updates/stable/helios/

No resultado, selecione JBoss Web and Java EE Development e clique em Next.

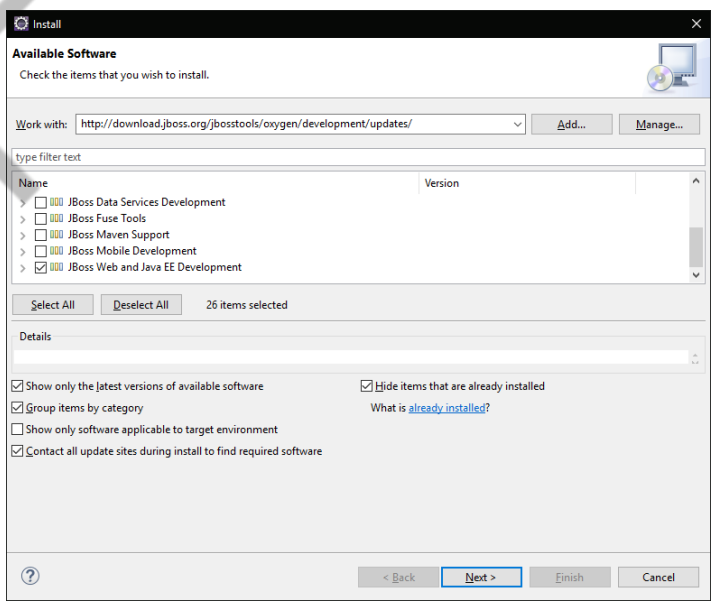

Figura 5.14 – Eclipse Available Software
Fonte: Elaborado pelo autor (2018)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Configure Install Remediation Page, conforme imagem abaixo e clique em Next.

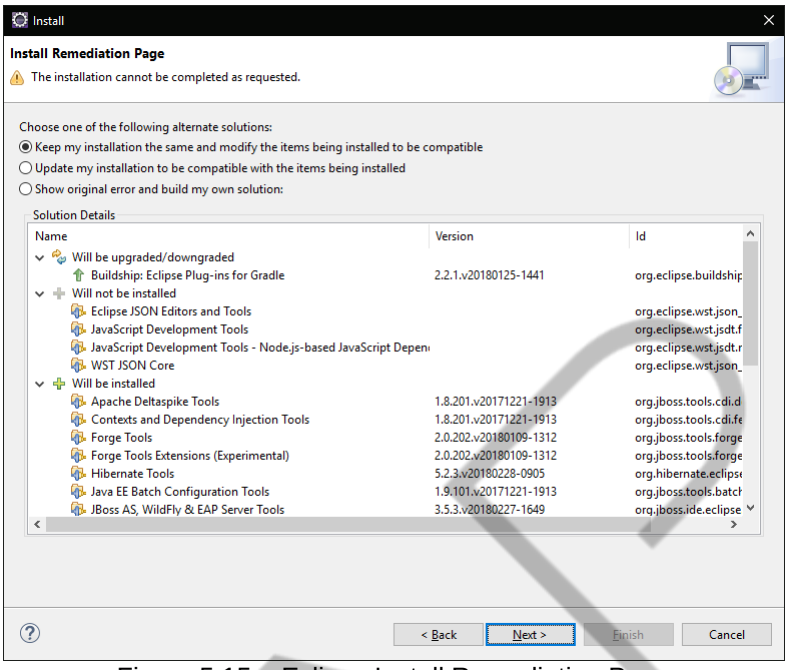

Figura 5.15 – Eclipse Install Remediation Page
Fonte: Elaborado pelo autor (2018)

Se desejar, dê uma olhada nos itens que serão instalados em JBoss AS, Wildfly & EAP Server Tools e clique em Next.

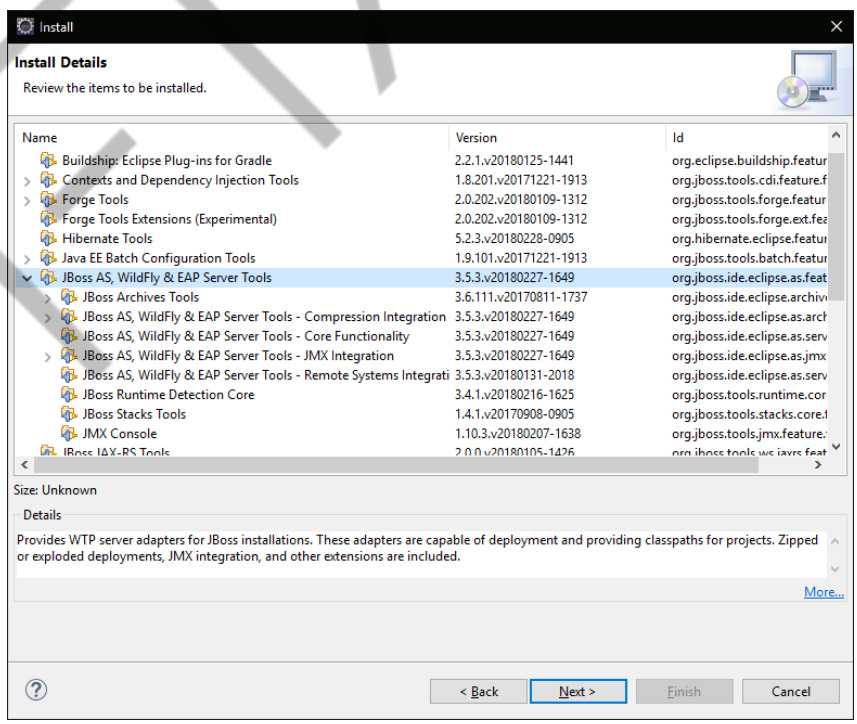

Figura 5.16 – Eclipse Install Details.
Fonte: Elaborado pelo autor (2018)

Aceite os termos e clique em Finish.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

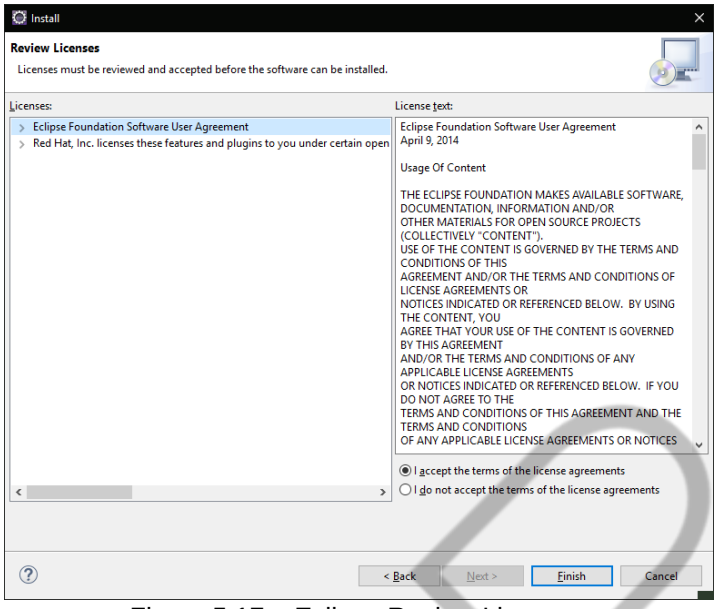

Figura 5.17 – Eclipse Review Licenses
Fonte: Elaborado pelo autor (2018)

O Eclipse será reiniciado.

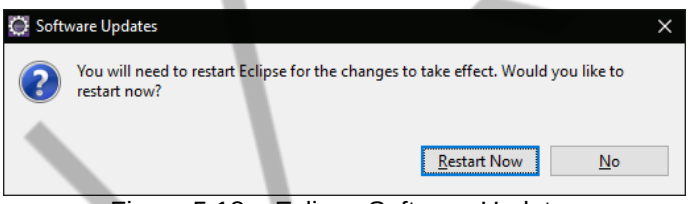

Figura 5.18 – Eclipse Software Updates
Fonte: Elaborado pelo autor (2018)

5.3.4.4 Criar um server adapter no projeto para o Wildfly

Na aba server, clique com o botão direito e escolha new server, procure o Wildfly 11 em JBoss Community e clique em Next.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

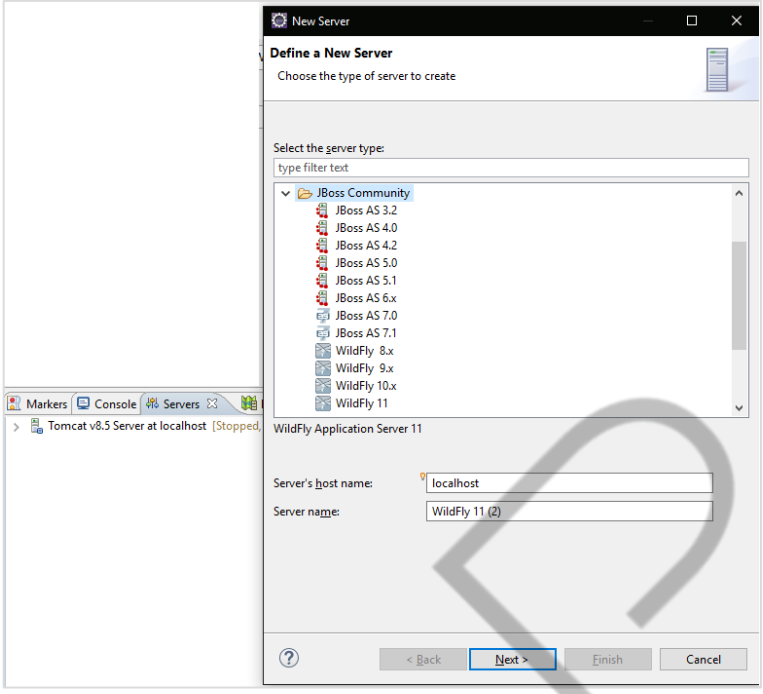

Figura 5.19 – Eclipse new server - Wildfly 11
Fonte: Elaborado pelo autor (2018)

Configure um New Server Adapter, conforme imagem abaixo e clique em Next.

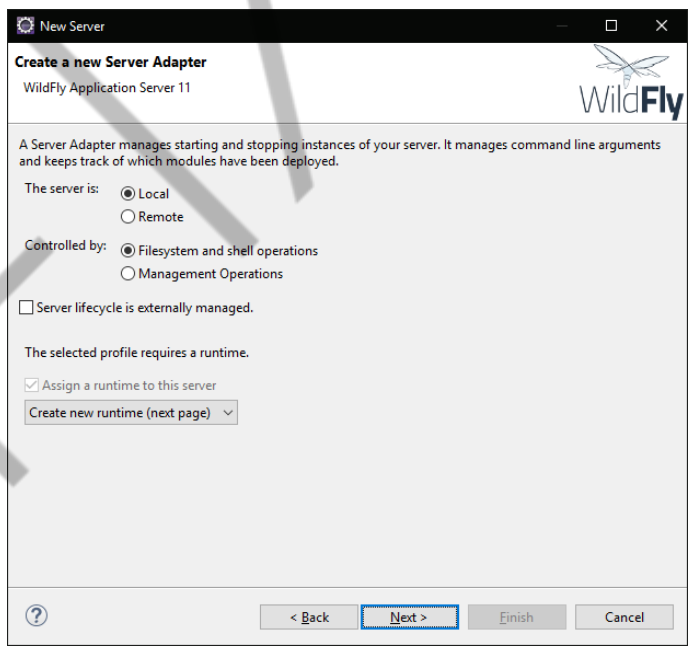

Figura 5.20 – Eclipse Create a new server adapter - Wildfly 11
Fonte: Elaborado pelo autor (2018)

Configure o Home Directory, com a pasta de instalação do seu Wildfly e clique em Finish.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

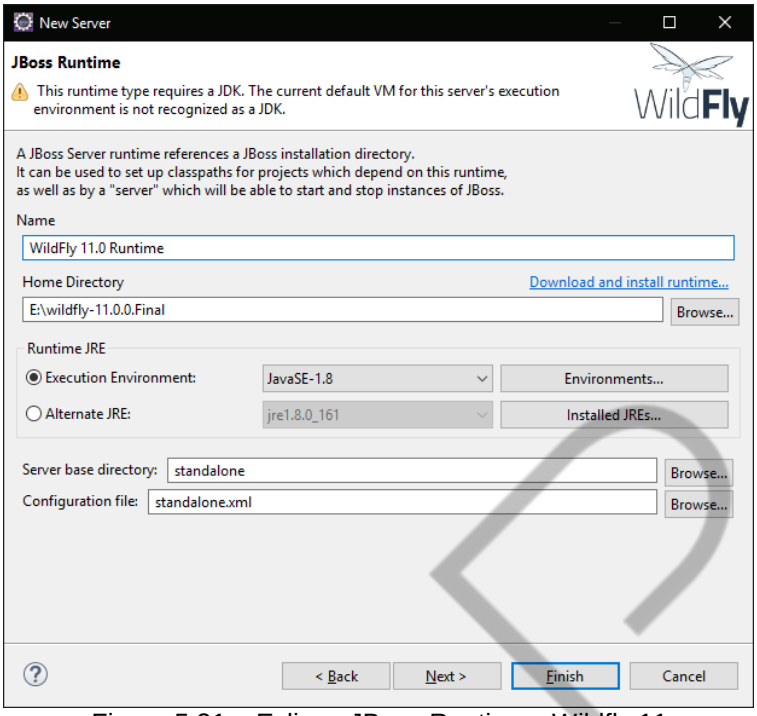

Figura 5.21 – Eclipse JBoss Runtime - Wildfly 11
Fonte: Elaborado pelo autor (2018)

Botão direito no Projeto JSF-Imoveis e em Server, selecione Wildfly 11 e clique em Apply and Close.

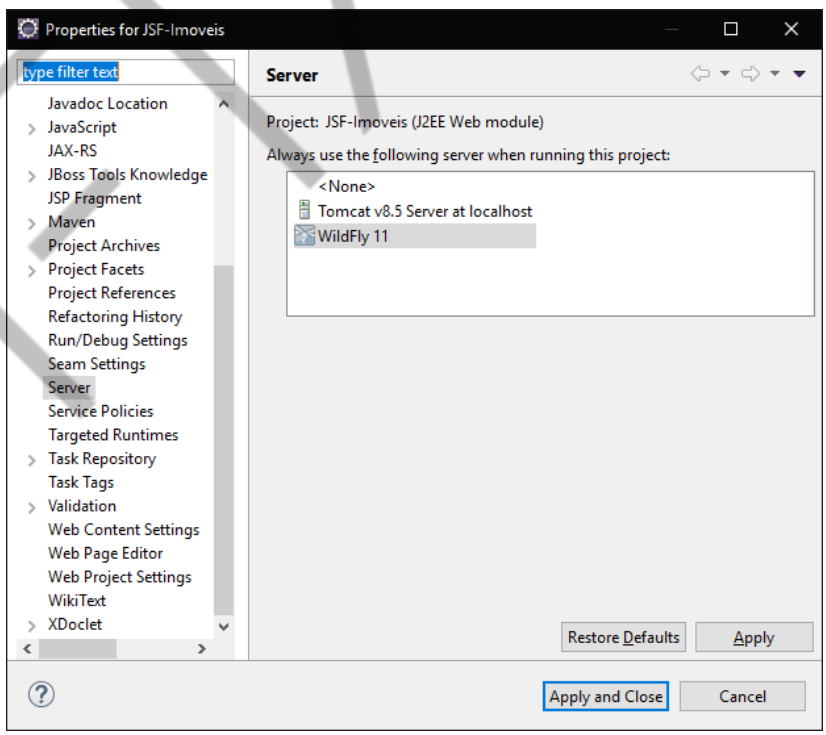

Figura 5.22 – Eclipse Server - Wildfly 11
Fonte: Elaborado pelo autor (2018)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Para indicar a raiz no nosso aplicativo ao Wildfly, no diretório WebContent / WEB-INF, crie o arquivo jboss-web.xml e codifique-o, conforme abaixo.

```
<?xml version="1.0" encoding="UTF-8"?>
<jboss-web>
    <context-root>JSF-Imoveis</context-root>
</jboss-web>
```

Código-fonte 5.14 – jboss-web.xml
Fonte: Elaborado pelo autor (2018)

5.3.4.5 Reconfigurar o POM da aplicação

Para utilizarmos as capacidades do JSF 2.3, vamos alterar a inclusão da biblioteca 2.2.2 de:

```
<dependency>
    <groupId>org.glassfish</groupId>
    <artifactId>javax.faces</artifactId>
    <version>2.2.2</version>
</dependency>
```

Código-fonte 5.15 – POM JSF 2.2.2
Fonte: Elaborado pelo autor (2018)

Para JEE API:

```
<dependency>
    <groupId>javax</groupId>
    <artifactId>javaee-api</artifactId>
    <version>8.0</version>
    <scope>provided</scope>
</dependency>
```

Código-fonte 5.16 – POM JSF 2.3
Fonte: Elaborado pelo autor (2018)

Essa API será utilizada em tempo de compilação apenas e proverá os recursos de que precisamos, tanto do JSF, quanto do CDI, veja https://javaee.github.io/javaee-spec/javadocs/.

Com o scope provided, não será incluída no empacotamento e ficará provida em tempo de execução pelo Wildfly, já que o escolhemos por ser um servidor de aplicação Full JEE. Selecione o projeto no Project Explorer do Eclipse e use o atalho ALT + F5, para que o Maven atualize o projeto.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

5.3.4.6 Reconfigurar o faces-config.xml

Altere o faces-config para a versão 2.3 do JSF.

```
<faces-config
      xmlns="http://xmlns.jcp.org/xml/ns/javaee"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
http://xmlns.jcp.org/xml/ns/javaee/web-facesconfig_2_3.xsd"
      version="2.3">
```

Código-fonte 5.17– faces-config.xml
Fonte: Elaborado pelo autor (2018)

5.3.4.7 Testar o projeto no Wildfly, backend gerenciado pelo JSF

Execute o projeto no Eclipse. A aplicação será executada no Wildfly.

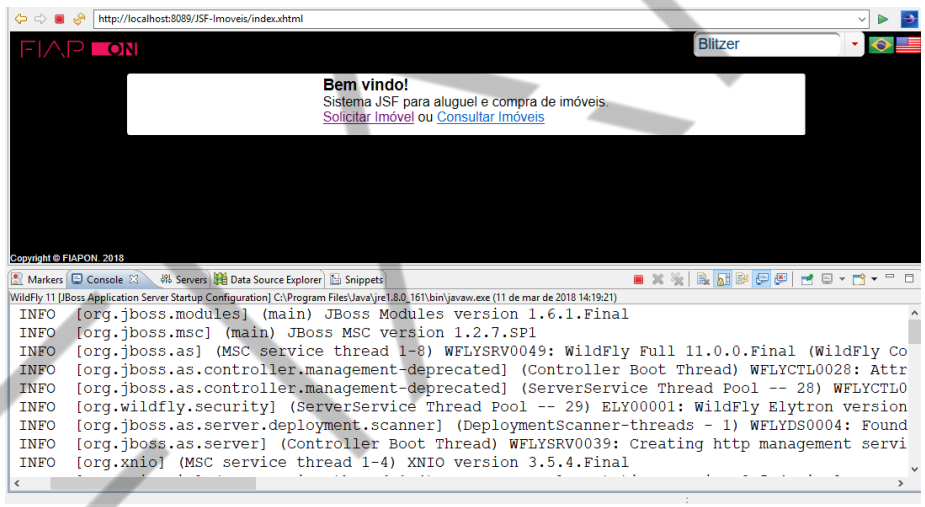

Figura 5.23 – index.xhtml
Fonte: Elaborado pelo autor (2018)

Como ainda não refatoramos os Beans, nessa execução continuam sendo gerenciados pelo container JSF.

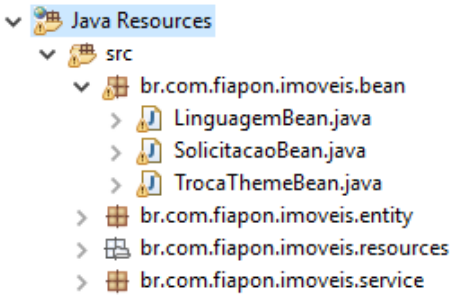

Figura 5.24 – Java Resources
Fonte: Elaborado pelo autor (2018)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Perceba que todos agora apresentam Warnings. A exemplo do LinguagemBean, ele e os demais estão como deprecated.

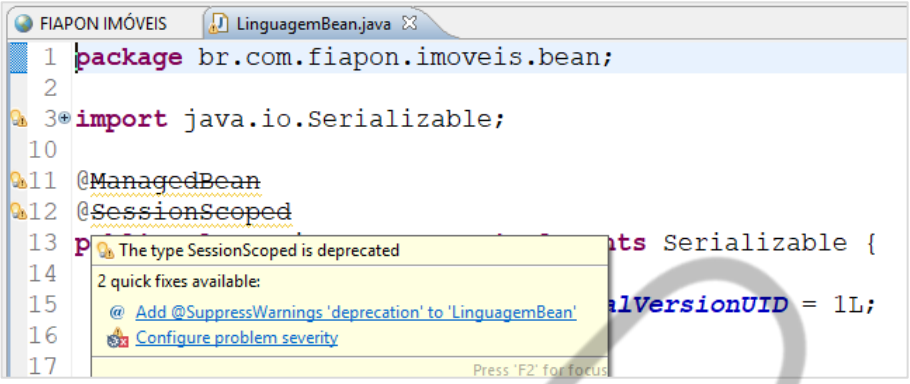

Figura 5.25 – Eclipse warnings
Fonte: Elaborado pelo autor (2018)

5.3.5 Refatorar os Beans

5.3.5.1 O LinguagemBean

Altere as anotações e os imports para:

```
package br.com.fiapon.imoveis.bean;

import java.io.Serializable;
import java.util.Locale;

import javax.annotation.PostConstruct;
import javax.enterprise.context.SessionScoped;
import javax.faces.context.FacesContext;
import javax.inject.Named;

@Named
@SessionScoped
public class LinguagemBean implements Serializable {
```

Código-fonte 5.18 – LinguagemBean.java
Fonte: Elaborado pelo autor (2018)

5.3.5.2 O SolicitacaoBean

Altere as anotações e os imports para:

```
package br.com.fiapon.imoveis.bean;

import java.util.ArrayList;
import java.util.List;
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
import java.util.Locale;
import javax.faces.application.FacesMessage;
import javax.faces.context.FacesContext;
import javax.inject.Inject;
import javax.inject.Named;

import br.com.fiapon.imoveis.entity.Solicitacao;

@Named
@RequestScoped
public class SolicitacaoBean implements Serializable{
```

Código-fonte 5.19 – SolicitacaoBean.java
Fonte: Elaborado pelo autor (2018)

5.3.5.3 O TrocaThemeBean

Altere as anotações e os imports para:

```
package br.com.fiapon.imoveis.bean;

import java.io.Serializable;
import java.util.List;

import javax.annotation.PostConstruct;
import javax.enterprise.context.SessionScoped;
import javax.inject.Inject;
import javax.inject.Named;

import br.com.fiapon.imoveis.entity.Theme;
import
  br.com.fiapon.imoveis.service.ThemePreferenceService;

@SessionScoped
@Named
public class TrocaThemeBean implements Serializable{

    private static final long serialVersionUID = 1L;
    //atributo com a lista de temas
    private List<Theme> themes;
    //Atributo com o nome do tema corrente
    private String theme;

    @Inject
    private ThemePreferenceService tp;
```

Código-fonte 5.20 – TrocaThemeBean.java
Fonte: Elaborado pelo autor (2018)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Remova o new do método init(), pois ThemePreferenceService será instanciado pelo CDI.

```
        @PostConstruct
        public void init() {
            tp = new ThemePreferenceService();
            //Busca no service todos os temas carregados do json
            themes = tp.getThemes();
            //Busca do service o tema default
            theme = tp.getTheme();
        }
```

Código-fonte 5.21– init()
Fonte: Elaborado pelo autor (2018)

5.3.5.4 O ThemePreferenceService

Essa entidade precisa ser descoberta pelo CDI, mas como um Bean da camada Model. A anotação @Named tem a finalidade de expor o Bean para a UI, portanto, não é boa prática empregá-la nessa classe.

Altere as anotações e os imports da ThemePreferenceService para:

```
package br.com.fiapon.imoveis.service;

import java.io.IOException;
import java.io.Serializable;
import java.nio.file.Files;
import java.nio.file.Paths;
import java.util.Arrays;
import java.util.List;

import javax.enterprise.inject.Model;
import javax.faces.context.FacesContext;

import com.google.gson.Gson;

import br.com.fiapon.imoveis.entity.Theme;

@Model
public class ThemePreferenceService implements Serializable{
```

Código-fonte 5.22 – ThemePreferenceService
Fonte: Elaborado pelo autor (2018)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

5.3.6 Testar o projeto no Wildfly, backend gerenciado pelo CDI

Execute o projeto no Eclipse. A aplicação será executada e agora, os Bean estão sob a responsabilidade do container CDI provido pelo servidor Full JEE Wildfly.

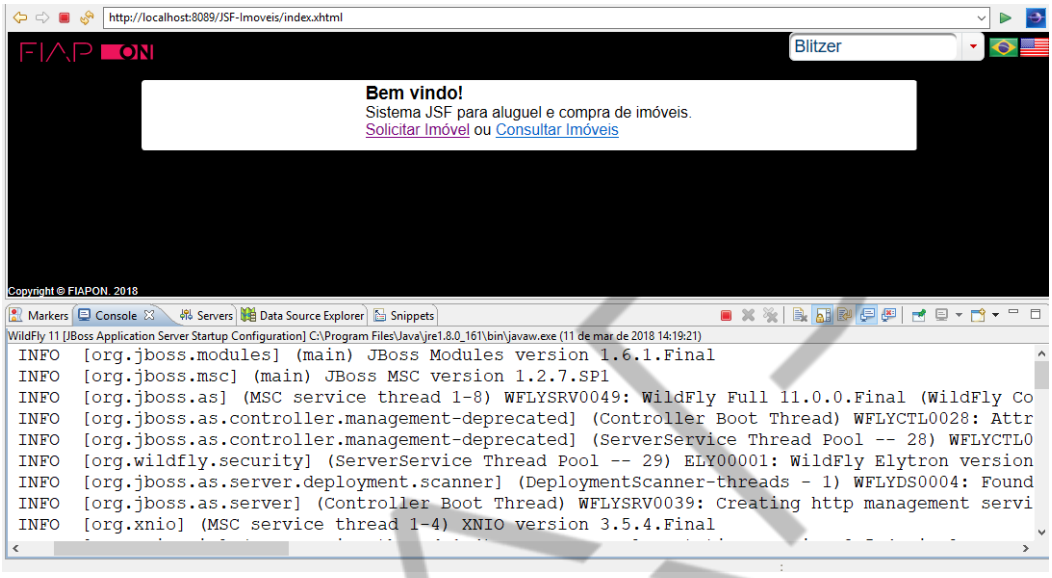

Figura 5.26 – index.xhtml
Fonte: Elaborado pelo autor (2018)

5.4 A aplicação JSF-Imoveis com JSF 2.3, CDI e JPA

5.4.1 Configurando o JPA

No Project Explorer, clique com o botão direito no projeto, selecione Properties e Project Facelets. Selecione JPA e clique Apply e Close.

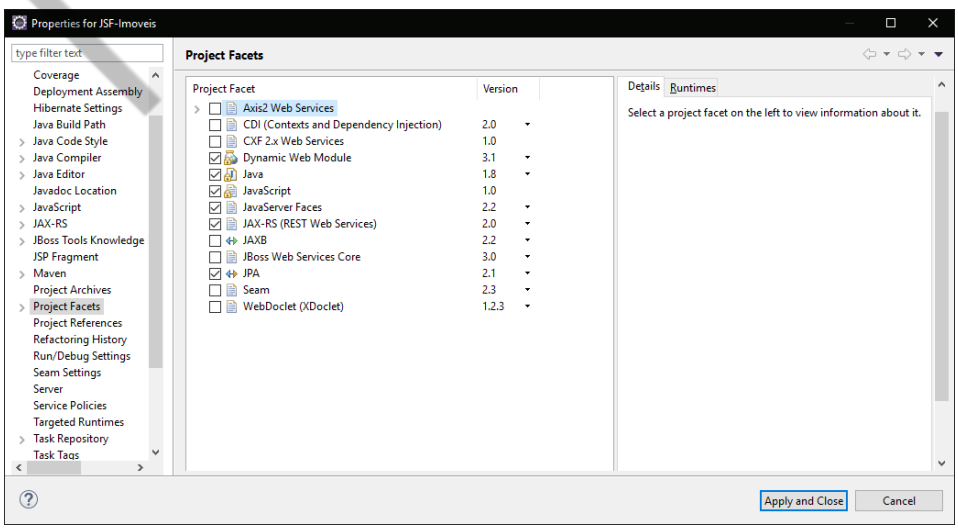

Figura 5.27 – Project Facets - JPA
Fonte: Elaborado pelo autor (2018)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

O arquivo persistence.xml será criado dentro da pasta META-INF. Codifique-o da forma abaixo. Não se esqueça de alterar o seu user e password.

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence version="2.1"
    xmlns="http://xmlns.jcp.org/xml/ns/persistence"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance"
    xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/per
sistence

    http://xmlns.jcp.org/xml/ns/persistence/persistence_2_1.
xsd">

    <persistence-unit  name="ORACLE_FIAP"  transaction-
type="RESOURCE_LOCAL">

    <provider>org.hibernate.jpa.HibernatePersistenceProvider
</provider>


    <class>br.com.fiapon.imoveis.entity.Solicitacao</class>

        <properties>
            <property        name="hibernate.show_sql"
value="true" />
            <property      name="hibernate.format_sql"
value="true" />
            <property    name="hibernate.hbm2ddl.auto"
value="update" />
            <property
name="javax.persistence.jdbc.driver"
value="oracle.jdbc.OracleDriver" />
            <property
name="javax.persistence.jdbc.user" value="PF0867" />
            <property
name="javax.persistence.jdbc.password" value="C7ABA2" />
            <property
name="javax.persistence.jdbc.url"

    value="jdbc:oracle:thin:@oracle.fiap.com.br:1521:orcl"/>
        </properties>
    </persistence-unit>
</persistence>
```

Código-fonte 5.23 – Persistence.xml
Fonte: Elaborado pelo autor (2018)

Não se esqueça, coloque o arquivo ojdbc6.jar dentro da pasta WebContent/WEB-INF/lib do projeto e adicione-a no Build Path.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

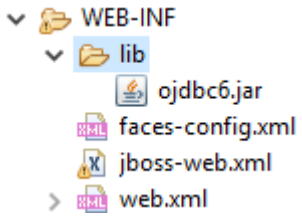

Figura 5.28 – ojdbc6.jar
Fonte: Elaborado pelo autor (2018)

5.4.2 Configurar o POM do projeto

Adicione as seguintes dependências do Hibernate, no POM do projeto.

```
<!-- JPA -->
 <dependency>
      <groupId>org.hibernate</groupId>
      <artifactId>
          hibernate-core
    </artifactId>
      <version>5.2.4.Final</version>
      <scope>provided</scope>
 </dependency>

 <dependency>
      <groupId>org.hibernate</groupId>
      <artifactId>
          hibernate-entitymanager
    </artifactId>
      <version>5.2.4.Final</version>
      <scope>provided</scope>
 </dependency>
```

Código-fonte 5.24 – POM JPA
Fonte: Elaborado pelo autor (2018)

5.4.3 As anotações da Entidade Solicitacao

Adicione as anotações de persistência. Como a nossa intenção não é criar um MER com todos os relacionamentos de um para muitos que essa tela possui, vamos anotar tais atributos com @ElementCollection. Desde a JPA 2.0, podemos utilizar essa anotação para persistir uma coleção, em nosso caso, os Lists com itens e cômodos. Essa é uma opção simples, mas não muito eficiente. Para o nosso exemplo, bastará.

```
package br.com.fiapon.imoveis.entity;
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
import java.io.Serializable;
import java.util.Calendar;
import java.util.List;

import javax.persistence.Column;
import javax.persistence.ElementCollection;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.SequenceGenerator;
import javax.persistence.Table;
import javax.persistence.Temporal;
import javax.persistence.TemporalType;


@Entity
@Table(name="TBL_IMV_SOLICITACAO")
@SequenceGenerator(name="sqSolicitacao",
sequenceName="SEQ_SOLICITACAO", allocationSize=1)
public class Solicitacao implements Serializable {

    private static final long serialVersionUID = 1L;

    @Id
    @GeneratedValue(strategy=GenerationType.SEQUENCE,
                                generator="sqSolicitacao")

    @Column(name = "cd_solicitacao")
    private int codigo;

    @Column(name = "nm_cidade")
    private String cidade;

    @Column(name = "ds_tipo")
    private String tipo;

    @Column(name = "ds_operacao")
    private String operacao;

    @Column(name = "ds_finalidade")
    private String finalidade;

    @Column(name = "vl_operacao")
    private float valor;

    @Column(name = "ds_imovel")
    private String descricao;

    @org.hibernate.annotations.Type(type="yes_no")
    @Column(name = "st_novo")
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
        private boolean novo;

        @ElementCollection
        @Column(name = "ds_itens")
        private List<String> itens;



        @ElementCollection
        @Column(name = "ds_comodos")
        private List<String> comodos;


        @Column(name = "vl_senha")
        private String senha;


        @Column(name = "ds_email")
        private String email;

        @Temporal(TemporalType.TIMESTAMP)
        @Column(name = "dt_solicitacao")
        private Calendar dataSolicitacao;

        public Solicitacao() {
             //Inicializa a data de solicitação
             dataSolicitacao= Calendar.getInstance();
        }

        public int getCodigo() {
             return codigo;
        }

        public String getTipo() {
             return tipo;
        }

        public void setTipo(String tipo) {
             this.tipo = tipo;
        }

        public String getFinalidade() {
             return finalidade;
        }

        public void setFinalidade(String finalidade) {
             this.finalidade = finalidade;
        }

        public String getOperacao() {
             return operacao;
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
        }

        public void setOperacao(String operacao) {
            this.operacao = operacao;
        }

        public void setCodigo(int codigo) {
            this.codigo = codigo;
        }

        public String getCidade() {
            return cidade;
        }

        public void setCidade(String cidade) {
            this.cidade = cidade;
        }

        public float getValor() {
            return valor;
        }

        public void setValor(float valor) {
            this.valor = valor;
        }

        public String getDescricao() {
            return descricao;
        }

        public void setDescricao(String descricao) {
            this.descricao = descricao;
        }

        public String getSenha() {
            return senha;
        }

        public void setSenha(String senha) {
            this.senha = senha;
        }

        public boolean getNovo() {
            return novo;
        }

        public void setNovo(boolean novo) {
            this.novo = novo;
        }

        public List<String> getItens() {
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
                return itens;
            }

            public void setItens(List<String> itens) {
                this.itens = itens;
            }

            public List<String> getComodos() {
                return comodos;
            }

            public void setComodos(List<String> comodos) {
                this.comodos = comodos;
            }

            public String getEmail() {
                return email;
            }

            public void setEmail(String email) {
                this.email = email;
            }

            public Calendar getDataSolicitacao() {
                return dataSolicitacao;
            }

            public void setDataSolicitacao(Calendar
dataSolicitacao) {
                this.dataSolicitacao = dataSolicitacao;
            }

            @Override
            public String toString() {
                return "Solicitacao [codigo=" + codigo + ",
cidade=" + cidade + ", tipo=" + tipo + ", operacao=" + operacao
                        + ", finalidade=" + finalidade + ",
valor=" + valor + ", descricao=" + descricao + ", novo=" + novo
                        + ", itens=" + itens + ", comodos=" +
comodos + ", senha=" + senha + ", email=" + email
                        + ", dataSolicitacao=" +
dataSolicitacao + "]";
            }


        }
```

Código-fonte 5.25 – Solicitacao.java
Fonte: Elaborado pelo autor (2018)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

5.4.4 Criar a Sequence SEQ_SOLICITCAO no Oracle FIAP

O atributo código será a chave primária da tabela e terá o valor incrementado por uma sequence do Oracle. Para tanto, conecte no Oracle FIAP com o SQL Developer e execute o seguinte comando.

```
CREATE SEQUENCE SEQ_SOLICITACAO INCREMENT BY 1 START WITH 1
MAXVALUE 99999999;
```

Código-fonte 5.26 – SEQ_SOLICITACAO
Fonte: Elaborado pelo autor (2018)

5.4.5 Adicionar as interfaces e classes para persistência

Da mesma forma que fizemos em outros capítulos, crie os pacotes e adicione as interfaces e classes genéricas que nos facilitam a persistência. Nossa aplicação ficará com a seguinte estrutura e arquivos.

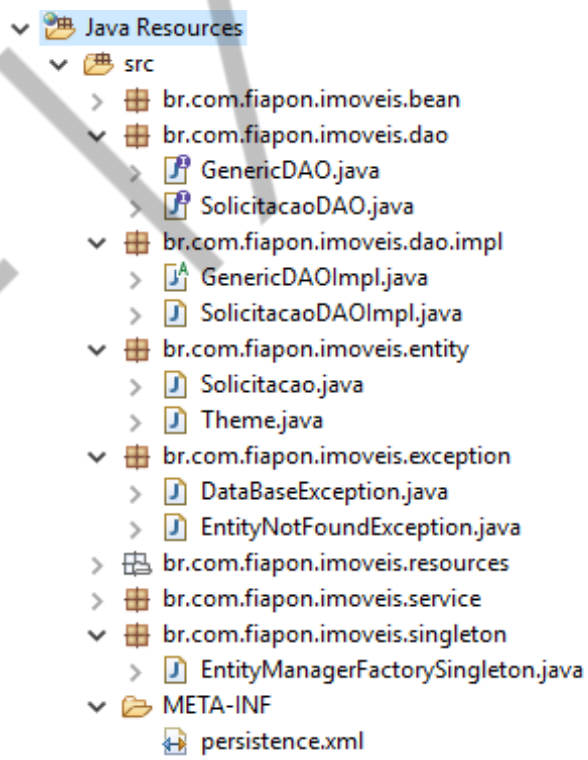

Figura 5.29 – Java Resources - JPA
Fonte: Elaborado pelo autor (2018)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

5.4.6 A interface SolicitacaoDAO

No pacote br.com.fiapon.imoveis.dao, codifique a interface SolicitacaoDAO.

```
package br.com.fiapon.imoveis.dao;

import br.com.fiapon.imoveis.entity.Solicitacao;

public interface SolicitacaoDAO
    extends GenericDAO<Solicitacao, Integer>{

}
```

Código-fonte 5.27 – SolicitacaoDAO.java
Fonte: Elaborado pelo autor (2018)

5.4.7 A classe SolicitacaoDAOImpl

No pacote br.com.fiapon.imoveis.dao.impl, codifique a classe SolicitacaoDAOImpl.

```
package br.com.fiapon.imoveis.dao.impl;

import br.com.fiapon.imoveis.dao.SolicitacaoDAO;
import br.com.fiapon.imoveis.entity.Solicitacao;

public class SolicitacaoDAOImpl extends
            GenericDAOImpl<Solicitacao, Integer>
            implements SolicitacaoDAO {
}
```

Código-fonte 5.28 – SolicitacaoDAOImpl.java
Fonte: Elaborado pelo autor (2018)

5.4.8 O Singleton EntityManagerFactorySingleton

No pacote br.com.fiapon.imoveis.singleton, garanta que `ORACLE_FIAP` seja o nome também utilizado no persistence-unit name, no arquivo persistence.xml

```
<persistence-unit name="ORACLE_FIAP" transaction-type="RESOURCE_LOCAL">
```

Código-fonte 5.29 – Trecho do persistence.xml
Fonte: Elaborado pelo autor (2018)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
package br.com.fiapon.imoveis.singleton;

import javax.persistence.EntityManagerFactory;
import javax.persistence.Persistence;

public class EntityManagerFactorySingleton {

    private static EntityManagerFactory fabrica;

    private EntityManagerFactorySingleton() {
    }

    public static EntityManagerFactory getInstance() {
        if (fabrica == null) {
            fabrica = Persistence
            .createEntityManagerFactory("ORACLE_FIAP");
        }

        return fabrica;
    }

}
```

Código-fonte 5.30 – EntityManagerFactorySingleton.java
Fonte: Elaborado pelo autor (2018)

5.4.9 Refatorar a classe SolicitacaoBean

Após imprimir todos os parâmetros, o método cadastrar precisa atualizar ou inserir uma nova solicitação, para tanto, faremos o uso de um if para verificar se o atributo código é maior que zero, se for, tratamos como atualização, senão, tratamos como uma nova solicitação.

Além disso, adicionamos a linha `this.solicitacao = new Solicitacao();` para que a tela, após a atualização ou cadastro, venha sem valores vinculados aos components.

```
// Cadastra a solicitação

public void cadastrar() {
    System.out.println(solicitacao.getCidade());
    System.out.println(solicitacao.getTipo());
    System.out.println(solicitacao.getOperacao());
    System.out.println(solicitacao.getFinalidade());
    System.out.println(solicitacao.getValor());
    System.out.println(solicitacao.getDescricao());
    System.out.println(solicitacao.getNovo());
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
        System.out.println(solicitacao.getItens());
        System.out.println(solicitacao.getComodos());
        System.out.println(solicitacao.getEmail());
        System.out.println(solicitacao.getSenha());
        System.out.println(solicitacao.getDataSolicitacao());

        SolicitacaoDAO solicitacaoDAO =
                                new SolicitacaoDAOImpl();

        if (this.solicitacao.getCodigo() > 0) {
            try {
                 solicitacaoDAO.atualizar(this.solicitacao);
            } catch (SQLIntegrityConstraintViolationException e)
           {
                            e.printStackTrace();
            } catch (DataBaseException e) {
                            e.printStackTrace();
            }
        } else {
            try {
                 solicitacaoDAO.adicionar(this.solicitacao);

            } catch (DataBaseException e) {
                 e.printStackTrace();
            }
        }

        this.solicitacao = new Solicitacao();

// Mensagem que será associada ao componente h:messages

        FacesMessage msg =
                 new FacesMessage("Imóvel Cadastrado!");

// Adiciona a menssagem para instância corrente da solicitação

        FacesContext.getCurrentInstance()
                        .addMessage(null, msg);

}
```

Código-fonte 5.31 – SolicitacaoBean.java
Fonte: Elaborado pelo autor (2018)

5.4.10 Testar o cadastro

Execute a aplicação e preencha o formulário.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Figura 5.30 – solicitacao.xhtml – Testando o cadastro
Fonte: Elaborado pelo autor (2018)

Clique em Enviar. A mensagem imóvel cadastrado será exibida.

Figura 5.31 – solicitacao.xhtml – Resultado do cadastro
Fonte: Elaborado pelo autor (2018)

Confirme o cadastro no SQL Developer.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

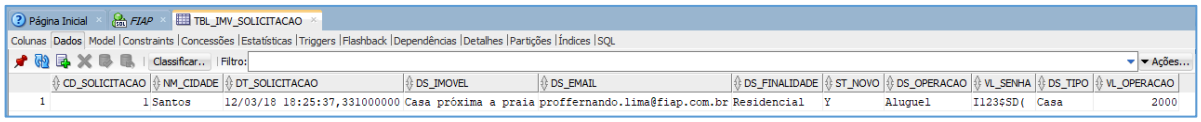

Figura 5.32 – Oracle Developer TBL_IMV_SOLICITACAO
Fonte: Elaborado pelo autor (2018)

Para as coleções, o Hibernate criou duas tabelas, (SOLICITACAO_COMODOS e SOLICITACAO_ITENS) e relacionou os itens e cômodos utilizando o código da solicitação. Por esse e outros motivos, comentamos que não é uma solução das melhores.

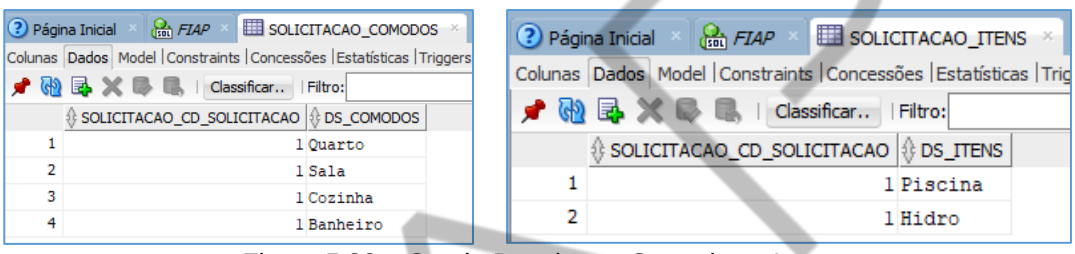

Figura 5.33 – Oracle Developer Comodos e Itens
Fonte: Elaborado pelo autor (2018)

Como não configuramos os nomes das tabelas, o padrão utilizado para criação é <Nome da Classe>_<Nome do Atributo>. Caso queira nomear as tabelas utilize `@CollectionTable(name =" ")`

5.4.11 Criar a tela de consulta

Copie e cole a tela construcao.xhml, renomeie o arquivo para consulta.xhtml. Vamos codificar a página com um objeto muito utilizado para esse tipo de funcionalidade, o DataTable. Esse componente será ligado a uma lista de elementos do SolicitacaoBean, por EL. Por sua vez, o SolicitacaoBean populará a lista, buscando os dados da tabela TBL_IMV_SOLICITACAO.

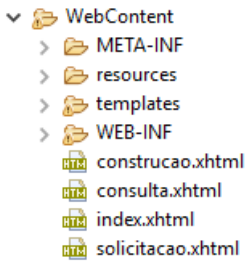

Figura 5.34 – consulta.xhtml
Fonte: Elaborado pelo autor (2018)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Antes de codificar a página, crie o seguinte método na SolicitacaoBean.

```
public List<Solicitacao> getSolicitacoes(){
      SolicitacaoDAO solicitacaoDAO
                      = new SolicitacaoDAOImpl();
      List<Solicitacao> solicitacoes = null;
      try {
          solicitacoes =  solicitacaoDAO.buscarTodos();
          solicitacoes =  solicitacaoDAO.buscarSolicitacoes();
      } catch (EntityNotFoundException e) {
              e.printStackTrace();
      }
        return solicitacoes;
      }
}
```

Código-fonte 5.32 – getSolicitacoes() - SolicitacaoBean.java
Fonte: Elaborado pelo autor (2018)

Codifique a tela consulta.xhtml. O DataTable será carregado com todas as solicitações, pois o atributo `value="#{solicitacaoBean.solicitacoes}"`, dispara o método getSolicitacoes, que acabamos de criar.

O DataTable possui outro importante atributo, o `var="solicitacoes"`. Será utilizado para que os valores retornados sejam exibidos na tabela. Como exemplo, o seguinte trecho cria uma coluna na tabela e exibe todas as cidades nessa coluna.

```
      <p:column>
          <f:facet name="header">Cidade</f:facet>
          <h:outputText value="#{solicitacoes.cidade}" />
      </p:column>
```

Código-fonte 5.33 – trecho consulta.xhtml
Fonte: Elaborado pelo autor (2018)

Neste exemplo, também utilizaremos a opção de paginação (`paginatorTemplate e rowsPerPageTemplate`) e a escolha das colunas dinamicamente, com o seguinte trecho de código:

```
<p:commandButton id="toggler" type="button" value="Colunas"
                                  icon="ui-icon-calculator" />
<p:columnTogglerdatasource="tabelaSolicitacoes"
                                  trigger="toggler" />
```

Código-fonte 5.34 – columnTogglerdatasource consulta.xhtml
Fonte: Elaborado pelo autor (2018)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Para que funcione, o id do p:dataTable deve ser o valor do p:columnTogglerdatasource.

Pois bem, vamos ao código da página consulta.xhtml.

```
<!DOCTYPE html>
<ui:composition template="/templates/_template.xhtml"
    xmlns="http://www.w3.org/1999/xhtml"
    xmlns:f="http://xmlns.jcp.org/jsf/core"
    xmlns:h="http://xmlns.jcp.org/jsf/html"
    xmlns:ui="http://xmlns.jcp.org/jsf/facelets"
    xmlns:p="http://primefaces.org/ui">

    <ui:define name="title">
        <h:outputText value="#{msg['Index.titulo']}" />
    </ui:define>

    <ui:define name="content">
        <div id="divContent">

            <h:form id="formTabela">

                <p:dataTable  id="tabelaSolicitacoes"
var="solicitacoes"

     value="#{solicitacaoBean.solicitacoes}" rows="10"
                emptyMessage="Solicitações   não
encontradas" paginator="true"

     paginatorTemplate="{CurrentPageReport}   {FirstPageLink}
{PreviousPageLink}  {PageLinks}  {NextPageLink}  {LastPageLink}
{RowsPerPageDropdown}"
                rowsPerPageTemplate="5,10,15">

                <f:facet name="header">
                Lista de Solicitações

                <p:commandButton        id="toggler"
type="button" value="Colunas"
                        icon="ui-icon-
calculator" />
                    <p:columnToggler
datasource="tabelaSolicitacoes" trigger="toggler" />
                </f:facet>

                <p:column>
                    <f:facet
name="header">ID</f:facet>
                    <h:outputText
value="#{solicitacoes.codigo}" />
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
                                </p:column>
                                <p:column>
                                     <f:facet
name="header">Cidade</f:facet>
                                     <h:outputText
value="#{solicitacoes.cidade}" />
                                </p:column>
                                <p:column>
                                     <f:facet
name="header">Tipo</f:facet>
                                     <h:outputText
value="#{solicitacoes.tipo}" />
                                </p:column>
                                <p:column>
                                     <f:facet
name="header">Operacao</f:facet>
                                     <h:outputText
value="#{solicitacoes.operacao}" />
                                </p:column>
                                <p:column>
                                     <f:facet
name="header">Finalidade</f:facet>
                                     <h:outputText
value="#{solicitacoes.finalidade}" />
                                </p:column>
                                <p:column>
                                     <f:facet
name="header">Valor</f:facet>
                                     <h:outputText
value="#{solicitacoes.valor}" />
                                </p:column>
                                <p:column>
                                     <f:facet
name="header">Descricao</f:facet>
                                     <h:outputText
value="#{solicitacoes.descricao}" />
                                </p:column>
                                <p:column>
                                     <f:facet
name="header">Novo</f:facet>
                                     <h:outputText
value="#{solicitacoes.novo}" />
                                </p:column>
                            </p:dataTable>
                        </h:form>
                    </div>
                </ui:define>
            </ui:composition>
```

Código-fonte 5.35 – consulta.xhtml
Fonte: Elaborado pelo autor (2018)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Na tela index.xhtml, troque o outcome de construçao para consulta.

```
<h:link outcome="consulta">
<h:outputText value="#{msg['Index.consultar.mensagem']}" />
</h:link>
```

Código-fonte 5.36 – consulta.xhtml
Fonte: Elaborado pelo autor (2018)

5.4.12 Testar a consulta

Execute a aplicação e navegue para a tela de consulta. A seguinte tela será exibida.

Figura 5.35 – consulta.xhtml em execução
Fonte: Elaborado pelo autor (2018)

Alterar para Bootstrap, o estilo será aplicado.

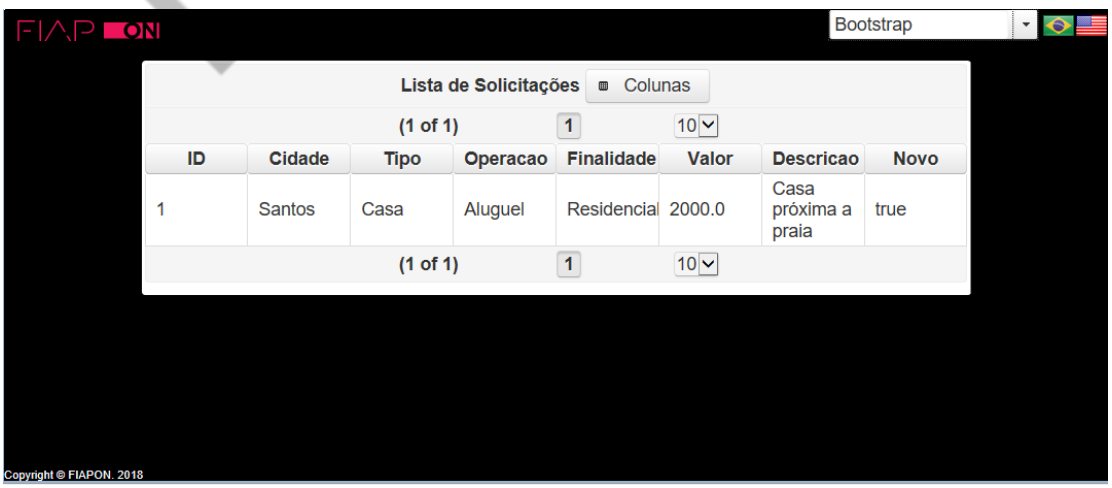

Figura 5.36 – consulta.xhtml com bootstrap
Fonte: Elaborado pelo autor (2018)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

5.5 Considerações finais

O objetivo deste conteúdo foi abordar o essencial sobre Primefaces, Temas, Ajax e JSF 2.3. Existem diversos pontos a serem explorados, como os diversos componentes do Primefaces, o maior emprego de CDI, a codificação completa do CRUD de solicitações, entre outros.

Portanto, sugerimos a leitura dos diversos conteúdos adicionais que são abordados nos livros e URLs de referência deste capítulo.

Bons estudos!

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

REFERÊNCIAS

HLAVATS, Ian. **Instant PrimeFaces Starter**. Packt Publishing, 2013.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

GLOSSÁRIO

JSF	Java Server Faces
JPA	Java Persistence API

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com